



Python 量化人的 Modern C++

从研究原型到可靠、可测量的生产系统

作者：Modern C++ 量化教程项目

组织：面向 Quant Developer 与 Research Engineer

时间：2026 年 7 月

版本：0.1

读者基础：已掌握 Python



先建立正确性，再讨论速度；先取得证据，再做优化。

前言

许多量化研究者第一次需要 C++，并不是因为 Python “不够好”，而是因为系统边界发生了变化：研究原型开始处理更大的数据，回测需要可重复的延迟，策略要接入已有的交易基础设施，或者团队希望把资源使用和失败方式变得可预测。本书要解决的，正是从“能用 Python 表达策略”到“能用 C++ 交付可靠量化组件”之间的工程距离。

本书假设你已经熟悉变量、函数、类、容器、异常、测试和虚拟环境。我们不会把一章花在解释循环是什么，而会追问：对象究竟在哪里，谁负责释放资源，复制发生在何时，接口承诺了什么，以及性能结论如何被测量证明。

贯穿全书的项目是一台小型事件驱动回测引擎。它不追求框架功能数量，而追求边界清楚：行情成为事件，策略产生订单意图，撮合器产生成交，组合模块更新状态，统计模块报告结果。每增加一个语言特性，都必须让这个系统更正确、更容易理解或更容易测量。

建议边读边运行代码。遇到性能章节时，先记录假设，再运行基准；遇到并发章节时，先画出所有权和停止协议，再写线程。真正有用的 Modern C++，不是特性清单，而是一套让成本、生命周期和契约变得可见的工具。

目录

前言	i
第一部分 从 Python 迁移	1
第一章 首次构建与诊断	2
1.1 本章输出：从空目录得到一行确定结果	2
1.2 第一份源文件：每个符号都是什么意思	2
1.2.1 场景：让操作系统找到程序入口	2
1.2.2 逐行读取程序结构	2
1.2.3 立即练习	3
1.3 单文件命令与构建流水线	3
1.3.1 从空目录运行 GCC 主线	3
1.4 三个故障实验：先分类，再修复	4
1.4.1 编译失败：缺少分号	4
1.4.2 链接失败：声明存在，定义缺失	4
1.4.3 运行期失败：程序主动报告启动错误	4
1.4.4 诊断记录模板：不要从最后一行猜原因	5
1.5 最小 CMake 目标与干净构建	5
1.5.1 Windows 差异只放在命令边缘	6
1.6 自测、编码任务与面试表达	6
1.7 本章小结	6
第二章 类型、表达式与控制流：写出第一个行情筛选器	7
2.1 本章任务：从五行行情得到一个确定答案	7
2.2 对象、类型与初始化	7
2.2.1 场景：一行数据需要四个有边界的对象	7
2.2.2 语法规则与最小实验	8
2.2.3 逐步观察：Python 名称不等于 C++ 对象	8
2.2.4 失败诊断与立即练习	9
2.3 表达式、auto 与显式转换	9
2.3.1 场景：价格乘数量还不够	9
2.3.2 语法规则与可运行路径	9
2.3.3 失败诊断与立即练习	10
2.4 条件表达式、if 与 switch	10
2.4.1 场景：有效买单必须同时通过多道门	10
2.4.2 布尔规则与短路求值	10
2.4.3 失败诊断与立即练习	11
2.5 循环与作用域	11
2.5.1 场景：已知行数与未知次数是两种循环	11
2.5.2 计数循环与对象可见范围	11
2.5.3 条件循环的完整练习答案	12

2.5.4 失败诊断与立即练习	13
2.6 整合：读取、筛选并汇总固定行情	13
2.6.1 按执行顺序走一遍	14
2.7 自测、练习与面试表达	14
2.8 本章小结	15
第三章 函数、引用、const 与多文件组织	16
3.1 本章任务：把循环中的计算拆成多文件工具	16
3.2 从表达式到函数	16
3.2.1 场景：同一公式需要一个名字和边界	16
3.2.2 重构路线：一次只移动一个边界	17
3.2.3 定义、参数、返回与局部作用域	17
3.3 按值、引用与可空指针	18
3.3.1 场景：函数是否可以改动调用者	18
3.3.2 指针：允许“没有观察对象”	20
3.4 重载：同名操作的不同参数表	20
3.5 头文件、实现文件与翻译单元	20
3.5.1 场景：让不同源文件共享同一接口	20
3.6 三个诊断实验：定义数量与对象寿命	22
3.6.1 缺失定义	22
3.6.2 重复定义	22
3.6.3 悬空引用	22
3.7 从干净目录复现与章节验收	22
3.8 自测、编码任务与面试表达	23
3.9 本章小结	23
第四章 STL、迭代器、算法与 ranges	24
4.1 先从 vector 开始	24
4.2 算法表达意图	24
4.3 ranges 是视图，不是新容器	24
4.4 关联查找与散列	25
4.5 数值归约的注意事项	25
4.6 常见错误	25
4.7 练习	25
4.8 本章小结	26
第二部分 写出可靠组件	27
第五章 类、接口与值语义：设计量化组件	28
5.1 类的价值是守住边界	28
5.2 组合优于继承	29
5.3 撮合与组合的公开契约	29
5.4 构造后即有效	30
5.5 接口深度	30
5.6 练习	31

5.7 本章小结	31
第六章 对象生命周期、RAII 与所有权	32
6.1 本章输出：让所有权状态可以被观察	32
6.2 对象生命周期、复制与移动	32
6.2.1 场景：一次行情缓冲区究竟销毁几次	32
6.2.2 完整实验与逐行轨迹	33
6.2.3 失败诊断与立即练习	34
6.3 RAII：把释放变成结构性结果	34
6.3.1 场景：提前返回也必须关闭报告文件	34
6.3.2 完整实验：文件流离开作用域后立即可读	34
6.3.3 失败诊断与立即练习	35
6.4 值、唯一所有权与共享所有权	35
6.4.1 选择顺序：先问能否直接拥有值	35
6.4.2 完整输出任务	35
6.4.3 失败诊断与立即练习	36
6.4.4 整合补充：把所有权意图写进函数接口	36
6.5 引用、裸指针与底层内存识读	37
6.5.1 场景：观察价格不等于拥有价格	37
6.5.2 完整实验：借用、C 数组与手动释放	37
6.5.3 失败诊断与立即练习	38
6.6 失败实验：让 AddressSanitizer 证明悬空	38
6.7 自测、练习与面试表达	39
6.8 本章小结	39
第七章 错误处理与可观测性	40
7.1 先分类，再选择机制	40
7.2 错误必须携带上下文	42
7.3 异常的正确位置	42
7.4 断言与验证	42
7.5 日志、指标与追踪	42
7.6 练习	43
7.7 本章小结	43
第八章 CMake、测试与代码质量	44
8.1 构建系统描述目标	44
8.2 测试公开行为	44
8.3 独立预期值	44
8.4 测试也会失效	44
8.5 质量工具分工	45
8.6 端到端样例	45
8.7 练习	46
8.8 本章小结	46

第三部分 量化性能工程	47
第九章 数据布局、缓存与分配	48
9.1 复杂度相同，耗时仍可不同	48
9.2 对象大小与对齐	50
9.3 分配是系统行为	50
9.4 false sharing	51
9.5 练习	51
9.6 本章小结	51
第十章 Benchmark、profiling 与优化方法	52
10.1 优化循环	52
10.2 微基准的陷阱	52
10.3 profiler 回答“时间在哪里”	52
10.4 端到端预算	52
10.5 性能与可维护性	52
10.6 练习	53
10.7 本章小结	53
第十一章 并发、原子与任务系统	54
11.1 先问为什么并发	54
11.2 不共享可变状态	54
11.3 互斥锁与原子	55
11.4 队列、背压与停止	55
11.5 任务系统	55
11.6 无锁不是默认目标	55
11.7 练习	55
11.8 本章小结	55
第十二章 数值计算与 Python 互操作	56
12.1 浮点不是实数	56
12.2 价格、收益与方差	57
12.3 先确认 Python 热点在哪里	57
12.4 pybind11 边界	57
12.5 批量边界胜过逐元素调用	58
12.6 练习	58
12.7 本章小结	58
第四部分 求职到工作	59
第十三章 贯穿项目：事件驱动回测引擎	60
13.1 从一个事件开始	60
13.2 运行项目	60
13.3 绩效统计契约	60
13.4 四个测试 seam	62
13.5 当前引擎明确不做什么	62

13.6 扩展顺序	63
13.7 性能报告怎么写	63
13.8 作为求职作品	63
13.9 练习	63
13.10 本章小结	63
第十四章 量化 C++ 求职与入职	64
14.1 先识别岗位	64
14.2 语言面试主线	64
14.3 代码题	64
14.4 系统设计题	64
14.5 项目如何写进简历	65
14.6 行为面试	65
14.7 入职后的前三个月	65
14.8 练习	65
14.9 本章小结	65
A 工具链速查	66
A.1 本书验证环境	66
A.2 构建 C++	66
A.3 构建 PDF	66
A.4 日志检查	66
附录 B 术语表	67
附录 C 练习答案与思路	68
C.1 第 1 章	68
C.2 第 2 章	68
C.3 第 3 章	69
C.4 第 4 章	70
C.5 第 5 章	70
C.6 第 6 章	70
C.7 第 7 章	71
C.8 第 8 章	71
C.9 第 9 章	71
C.10 第 10 章	71
C.11 第 11 章	71
C.12 第 12 章	71
C.13 第 13 章	72
C.14 第 14 章	72
附录 D 推荐阅读与 30 天路线	73
D.1 推荐阅读	73
D.2 第 1 周：语言与所有权	73
D.3 第 2 周：接口与可靠性	73
D.4 第 3 周：性能与并发	73
D.5 第 4 周：项目与求职	73

D.6 完成判据	74
参考文献	75

第一部分

从 Python 迁移

第一章 首次构建与诊断

内容提要

- ❑ 从空目录写出、编译并运行第一个 C++ 程序
- ❑ 逐个读懂 `#include`、`main`、花括号、分号和标准输出
- ❑ 区分源文件、目标文件、可执行文件以及编译与链接
- ❑ 用最小 CMake 目标完成一次全新的目录外构建
- ❑ 通过三个隔离实验辨认编译、链接和运行期故障

注 先修知识：会在终端进入目录、用文本编辑器保存文件，并能运行 Python 脚本；不要求任何 C++ 语法或编译经验。

Python 迁移边界：Python 可以直接运行 `python script.py`；C++ 源文件必须先变成当前平台的可执行文件。

常见错误与诊断：先记录完整命令和退出状态，再判断失败发生在编译、链接还是程序已经启动之后。

1.1 本章输出：从空目录得到一行确定结果

量化开发的第一个交付物不应是“编辑器里看起来像代码”，而是一条别人可以复现的构建链。本章任务从一个空目录开始，创建 `first_program.cpp` 与 `CMakeLists.txt`，最终得到：

```
quant-cpp-ready
```

完成判据有四项：单文件 GCC 命令能构建并运行；CMake 能在全新的 `build/` 目录配置和构建；程序精确输出上述文本并以状态 0 结束；你能仅凭命令停在哪一步，把三个故障实验归为编译、链接或运行期。

1.2 第一份源文件：每个符号都是什么意思

1.2.1 场景：让操作系统找到程序入口

Python 文件可以从第一条顶层语句开始执行。C++ 可执行程序需要一个约定入口，操作系统启动进程后由运行时进入名为 `main` 的函数。先完整保存下面的真实源码，不要省略标点：

Listing 1.1: 第一个可运行的 C++ 程序

```
1 #include <iostream>
2
3 int main() {
4     // Keep the first observable result exact and easy to verify.
5     std::cout << "quant-cpp-ready\n";
6     return 0;
7 }
```

1.2.2 逐行读取程序结构

`#include <iostream>` 是预处理指令。行首的 `#` 表示它在正式编译前处理；尖括号中的 `iostream` 是标准库头文件名，它让当前源文件看见标准输入输出工具的声明。这一行末尾没有分号，因为它不是 C++ 语句。

`int main()` 声明程序入口。这里先把 `int` 理解为“该入口会交回一个整数状态”，`main` 是标准规定的名称，空圆括号表示本版本暂不接收命令行参数。左花括号 `{` 开始函数体，匹配的右花括号 `}` 结束函数体；缩进帮助人阅读，但花括号才决定结构。

`//` 从当前位置开始一行注释；`/*` 与 `*/` 则界定可以跨行的块注释。编译器不会把注释内容当作要执行的语句，但块注释不能嵌套：遇到第一个 `*/` 就已经结束，因此不要用它包住一段本身含块注释的代码。`std::cout` 是标准输出流；`std::` 指明名称属于标准库命名空间，双冒号是作用域解析符。`<<` 把右侧内容送入输出流，双引号包住要输出的文本，`\n` 表示换行。字符串和数字的正式类型留到第 2 章，本章只要求能正确识读这条输出语句。

行尾的分号结束一条 C++ 语句。`return 0;` 把成功状态交回运行环境；在 Linux/WSL 中，状态 0 约定为成功，非零表示失败。分号、圆括号、花括号和引号都是语法的一部分，不能像 Python 中某些可选标点那样随意删去。

注 Python 迁移提示：Python 的缩进同时承担语法结构，`print()` 由运行时按名称查找。C++ 用花括号划分函数体，用分号结束语句，并在编译阶段解析 `std::cout`。两者都能向终端输出文本，但程序如何到达“可运行”状态并不等价。

1.2.3 立即练习

先只把源码中的 `//` 注释改成单行的 `/* ... */` 注释，预测输出应逐字符不变，再构建运行；不要嵌套块注释。然后只把输出文本改为 `quant-cpp-learning`，预测哪一行会变化并再次运行。若只改文本却出现语法错误，优先检查成对双引号、末尾分号和 `\n` 是否仍在；完成后恢复章首约定输出，确保精确测试重新通过。

1.3 单文件命令与构建流水线

1.3.1 从空目录运行 GCC 主线

在 Linux 或 WSL 中创建目录，用编辑器把清单保存为 `first_program.cpp`，然后运行：

```
mkdir ch01-first-build
cd ch01-first-build
g++ -std=c++20 -Wall -Wextra -Wpedantic \
    first_program.cpp -o first_program
./first_program
echo $?
```

`g++` 是 GCC 的 C++ 驱动程序；`-std=c++20` 选择本书语言版本；三个警告选项要求编译器报告常见可疑代码；`-o first_program` 指定输出文件名。反斜杠位于 shell 行末时表示命令在下一行继续，它不是源文件内容。`./` 明确运行当前目录中的文件，`echo $?` 查看刚结束程序的状态，应为 0。

一条 `g++` 命令隐藏了多步流水线：

source → preprocess → compile → object file → link → executable → run.

预处理展开 `#include` 等指令；编译器检查当前翻译单元并产生目标文件；链接器把目标文件和所需库组合为可执行文件；只有最后一步才真正启动你的程序。想观察中间边界，可把命令拆开：

```
g++ -std=c++20 -Wall -Wextra -Wpedantic \
    -c first_program.cpp -o first_program.o
g++ first_program.o -o first_program
./first_program
```

`-c` 表示只编译、不链接，所以第一条命令成功只能证明目标文件已经生成，不能证明最终程序的所有名称都有定义。

注Python 迁移提示：导入 Python 模块也可能失败，但普通 Python 工作流不会先生成目标文件再调用链接器。C++ 的阶段边界使一部分错误更早暴露，也要求你看清究竟是哪条命令失败，而不是把所有诊断统称为“编译不过”。

1.4 三个故障实验：先分类，再修复

故意失败源码都隔离在 `code/ch01/failures/`，不进入默认绿色构建。每个实验只验收稳定类别和非零状态，不绑定某一版 GCC 的完整英文句子。

1.4.1 编译失败：缺少分号

`missing_semicolon.cpp` 的输出语句末尾故意少了分号：

```
1 std::cout << "quant-cpp-ready\n"
2 return 0;
```

运行 `g++ -std=c++20 -c code/ch01/failures/missing_semicolon.cpp`。编译器会在读到 `return` 附近报告“期望分号”类别；箭头位置可能落在下一行，因为解析器直到看到下一个记号才确认上一条语句没有结束。根因是语法不完整，修复是在输出语句后恢复分号。立即练习：只删除 `return 0;` 后的分号，比较首个诊断位置，然后恢复文件。

1.4.2 链接失败：声明存在，定义缺失

`missing_definition.cpp` 包含一个函数声明和一次调用，但项目没有提供函数体：

```
1 double expected_notional(); // declaration only
2 // main later calls expected_notional()
```

先用 `-c` 生成目标文件会成功，因为当前文件已经知道调用的名称和返回类型；再运行：

```
g++ missing_definition.o -o missing_definition
```

链接器应报告 `undefined reference` 类别。函数声明与定义会在第 3 章从零教学；此处只建立诊断规则：能产生 `.o` 但不能产生可执行文件，问题就在链接阶段。修复是提供且参与链接的唯一定义，而不是反复修改编译警告选项。立即练习：分别记录两条命令的退出状态，确认前者为 0、后者非零。

1.4.3 运行期失败：程序主动报告启动错误

`startup_failure.cpp` 可以成功编译和链接，但启动后把错误写到 `std::cerr`，再 `return 2;`

```
1 std::cerr << "startup-error: market-data path is required\n";
2 return 2;
```

这模拟量化程序在启动检查中发现缺少行情路径。`std::cerr` 是标准错误流，便于把诊断与正常结果分开；状态 2 是本实验定义的失败协议。若可执行文件已经启动并打印上述消息，故障阶段就是运行期，不是链接。立即练习：只把状态改为 3，运行后用 `echo $?` 验证；随后恢复为 2，使自动验收重新通过。

1.4.4 诊断记录模板：不要从最后一行猜原因

编译器和链接器可能在一个根因后产生许多级联消息。可靠记录按固定顺序写五项：当前目录与完整命令；第一条非零退出的命令；失败前最后生成的产物；第一条能够解释后续消息的诊断；修复后重跑的命令和结果。只截取终端末尾一句，常会丢掉真正的文件名、行号和阶段。

阶段也能由产物反推：没有目标文件，先查预处理或编译；已有 .o 而没有可执行文件，查链接；已有可执行文件并且程序已经输出诊断，查运行期输入或业务状态。若旧产物仍在，先删除它或换用全新构建目录，避免把昨日成功生成的文件误当成今日命令的证据。

立即练习：为三个实验各写一行“命令 → 最后产物 → 退出状态 → 稳定类别”。三行应分别停在源码、目标文件和已启动进程处；若分类只写“报错”，还没有完成诊断。

1.5 最小 CMake 目标与干净构建

手写 GCC 命令适合学习阶段边界；项目增长后，应用 CMake 描述“要构建什么”。下面的文件与第一个程序放在同一目录，并参与本项目的干净构建测试：

Listing 1.2: 第一个最小 CMake 项目

```
1 cmake_minimum_required(VERSION 3.20)
2 project(ch01_first_build LANGUAGES CXX)
3
4 set(CMAKE_CXX_STANDARD 20)
5 set(CMAKE_CXX_STANDARD_REQUIRED ON)
6 set(CMAKE_CXX_EXTENSIONS OFF)
7
8 add_executable(ch01_first_program first_program.cpp)
```

cmake_minimum_required 声明最低 CMake 版本；project 建立一个使用 C++ 的项目；三个 set 选择 C++20、要求编译器支持它并关闭厂商扩展；add_executable 创建名为 ch01_first_program 的目标，并把源文件交给它。

在包含这两个文件的项目目录中运行：

```
cmake -S . -B build
cmake --build build --target ch01_first_program
./build/ch01_first_program
```

-S . 指定源目录，-B build 指定目录外构建树；源码与生成文件因此不会混在一起。第一次运行前 build/ 不应存在，这叫干净构建。若只删除可执行文件而保留旧缓存，不能证明项目从零可配置。验收完成后可删除整个 build/，再重复三条命令。

注Python 迁移提示：Python 项目常用 pyproject.toml 描述包、用虚拟环境隔离依赖，并由解释器直接运行入口；这里的 CMakeLists.txt 描述原生构建目标。cmake -S . -B build 只读取项目并生成当前平台的构建系统，不会运行程序；cmake --build build 才调用编译器与链接器。CMake 因而更像跨平台的构建描述层，不是 C++ 解释器或包管理器。

做一次只有一个变量的失败练习：在项目副本里只把 add_executable 中的 first_program.cpp 改为不存在的 missing.cpp，并把配置输出写入全新的 build-broken/。预期 cmake -S . -B build-broken 在配置或生成阶段以非零状态结束，稳定类别是 Cannot find source file，而且不会产生可运行目标。把文件名恢复后删除 build-broken/，从空构建目录重跑配置、构建和程序；若仍失败，先核对源目录是否同时包含清单中的两个文件，再读第一条 CMake 诊断，而不是改编编译器选项。

1.5.1 Windows 差异只放在命令边缘

正文主线使用 Linux/WSL 与 GCC。在 PowerShell 中，CMake 的前两条命令保持不变；单配置生成器的程序通常写作 `.\build\ch01_first_program.exe`，Visual Studio 等多配置生成器可能位于 `build\Debug`。直接使用 MSVC 时警告选项是 `/W4 /permissive-`，而不是 GCC 的 `-Wall -Wextra -Wpedantic`。不要维护两套教学流程：以 `cmake --build` 屏蔽生成器细节，需要 Linux sanitizer 或书中精确 shell 命令时使用 WSL。

1.6 自测、编码任务与面试表达

1. `#include <iostream>` 在哪一阶段处理？为什么行末没有分号？
2. `int main()` 的名称、圆括号、花括号和返回状态分别承担什么职责？
3. `std::cout << "text\n";` 中，`std::`、`<<`、引号、转义和分号分别表示什么？
4. 单条 `g++` 命令隐藏了哪些阶段？`-c` 为什么能隔开编译与链接？
5. 缺分号、缺定义和状态 2 分别在哪个阶段失败？每种修复应改变什么？
6. CMake 的源目录和构建目录为什么应分开？怎样证明一次构建是干净的？
7. Windows 与 Linux 主线最值得记录的三个命令差异是什么，哪些心智模型不变？

编码任务：

1. 从空目录重建两个文件，先用 GCC 单文件命令得到 `quant-cpp-ready`，再用 CMake 在全新 `build/` 中得到同样结果。
2. 依次运行三个故障实验，为每个实验记录失败命令、阶段、非零状态、稳定诊断类别、根因和恢复步骤。
3. 在第一个程序中分别增加一行 `//` 注释和一段不嵌套的 `/* ... */` 注释，证明输出完全不变；再故意删除输出语句的分号并恢复。

问题 1.1 面试检查点 用 90 秒回答：“从 `first_program.cpp` 到正在运行的进程发生了什么？”回答必须按预处理、编译、链接、启动排序，并各给一个可观察产物或失败类别；最后说明为什么“编译成功”不一定意味着“程序可运行且业务成功”。

1.7 本章小结

第一个 C++ 程序已经建立最小闭环：头文件提供声明，`main` 提供入口，花括号和分号构成语法结构，标准流产生可观察输出，状态码向环境报告成功或失败。GCC 把源码经过预处理、编译和链接变成可执行文件，CMake 则把同一目标描述为可重复的目录外构建。诊断时不要只读最后一行错误：先确认停在哪条命令、是否已有目标文件、是否已有可执行文件、程序是否已经启动，再处理该阶段的首个根因。

第二章 类型、表达式与控制流：写出第一个行情筛选器

内容提要

- ❑ 把 Python 的“名称绑定对象”心智模型切换为 C++ 的“声明对象并初始化”
- ❑ 选择基本类型、定宽整数、字面量和 auto，并识别窄化
- ❑ 用算术、比较、布尔表达式和显式转换计算行情指标
- ❑ 用 if/else、switch、for 和 while 控制执行路径
- ❑ 从标准输入读取固定行情，筛选有效买单并汇总名义金额

注 先修知识：已完成第 1 章，能在 Linux/WSL 中编译、链接并运行单文件程序；本章不要求会自定义函数、容器、类、引用或指针。

Python 迁移边界：Python 变量是可重新绑定的名称；C++ 声明会创建具有固定类型和作用域的对象，复制一个基础类型对象会得到独立的值。

常见错误与诊断：先区分编译诊断与运行结果，再检查初始化、整数除法、隐式转换、分支边界和循环是否终止。

2.1 本章任务：从五行行情得到一个确定答案

研究原型里，一条筛选可能只是几行 `DataFrame` 表达式；面试白板题或低延迟数据入口却常要求你明确说明：每个字段是什么类型，坏数据走哪条分支，循环何时停止，累计值如何更新。本章的输出任务只使用基础语法，不借助尚未教学的自定义函数、结构体或容器。

程序从标准输入读取一个行数和五行数值行情。每行依次是“证券编号、方向代码、价格、数量”；方向 1 表示买，2 表示卖，其他代码无效。程序只选中编号、价格、数量均为正的买单，然后打印选中数、拒绝数、名义金额总和与每条选中记录的平均名义金额。

完成本章后，下面的输入必须得到逐字符一致的核心数值结果：

```
5
101 1 100.5 10
102 2 50.0 4
103 1 25.0 0
104 1 10.25 3
105 9 12.0 5
```

```
selected=2 rejected=3 buy_notional=1035.75 average=517.875
```

这就是本章的验收标准：不是“看懂语法”，而是能在不复制最终答案的前提下重写程序、解释每个对象的状态变化，并用给定数据复现输出。

2.2 对象、类型与初始化

2.2.1 场景：一行数据需要四个有边界的对象

读入一行行情时，程序需要保存证券编号、方向、价格和数量。如果这些值没有确定的初始状态，后面的筛选就可能读取垃圾值；如果价格被随意塞进整数，小数部分会悄悄丢失。C++ 因而要求先回答“创建什么类型的对象”，再讨论计算。

2.2.2 语法规则与最小实验

最常见的声明由“类型、对象名、初始化器、分号”组成。下面不是省略外壳的片段，而是参与 C++20 构建并由 CTest 检查输出的完整程序：

Listing 2.1: 对象、字面量与独立复制

```

1  #include <cstdint>
2  #include <iostream>
3
4  int main() {
5      int side_code{1};
6      double price{100.5};
7      std::int64_t quantity{10};
8      bool is_buy{true};
9      char venue{'X'};
10
11     int original{10};
12     int copied{original};
13     original = 11;
14
15     std::cout << "side=" << side_code << " price=" << price
16               << " quantity=" << quantity << " buy=" << is_buy
17               << " venue=" << venue << " changed=" << original
18               << " copied=" << copied << '\n';
19 }
```

`int` 保存整数，`double` 保存双精度浮点数，`bool` 只有 `true` 和 `false` 两种逻辑状态，`char` 保存一个字符。`std::int64_t` 来自 `<cstdint>`，在提供它的平台上恰有 64 位；当数量或时间戳需要明确位宽时，它比“希望 `long` 足够大”更可检查。

花括号 `{}` 同时标记对象从这里开始生命周期，并拒绝一部分会丢信息的窄化。空花括号执行值初始化：`int selected{}`；得到零，`bool is_buy{}`；得到 `false`。相反，局部基础类型 `int selected`；没有可靠的业务初值；读取它不是“随机但可用”，而是程序错误。

字面量本身也有类型。1 通常是 `int`，1.0 是 `double`，`true` 是 `bool`，单引号中的 `'X'` 是字符字面量，双引号中的 `"side="` 是字符串字面量。`'\n'` 表示换行字符，它与包含两个可见字符的 `"\n"` 不同。因此 `double price{100}`；可以精确表示该值，而 `int price{100.5}`；会在编译阶段被拒绝。

为了让本章不依赖读者猜测程序外壳，这里逐项说明：`#include <cstdint>` 与 `#include <iostream>` 让当前源文件看见所需标准库声明；`int main()` 定义程序入口，`int` 是返回状态的类型，空圆括号表示这个版本不接收命令行参数，随后的一对花括号包住入口函数体。`std::cout` 是标准输出流，`<<` 依次把右侧的字符串字面量和对象值送入流。`std::` 指明标准库名称所在的命名空间。最后一个右花括号结束 `main`；走到这里等价于成功返回状态 0。第 3 章才会系统教学如何定义和调用自己的函数。

2.2.3 逐步观察：Python 名称不等于 C++ 对象

在 Python 中，赋值把名称绑定到对象；两个名称可以指向同一个可变列表：

```

1  a = [10]
2  b = a
3  a[0] = 11
4  print(b[0])          # 11
```

在上面的完整 C++ 程序中，`original` 与 `copied` 是两个独立的 `int` 对象。`copied{original}` 用 `original` 当时的值初始化新对象，之后把 `original` 赋值为 11，不会改动 `copied`；可观察输出同时包含 `changed=11` 与 `copied=10`。

等号在对象创建之后出现时是赋值，不是初始化。对象的类型仍是 `int`，不能像 Python 名称那样下一行重新绑定成字符串。共享和借用当然可以在 C++ 中表达，但要用后续章节才教学的引用、指针或拥有型对象，不能从普通按值声明中猜出来。

注Python 迁移提示：两种语言都允许把已有值用于新变量；边界在于 Python 名称通常保存对象引用，而本章的 C++ 基础类型声明创建独立对象。Python 类型标注通常不改变运行期绑定规则，C++ 对象类型则直接限制初始化、运算和存储表示。

2.2.4 失败诊断与立即练习

故意失败文件 `code/ch02/failures/narrowing.cpp` 包含如下核心语句：

```
1 int rounded_price{100.5}; // intentional compile failure
```

用 `g++ -std=c++20 code/ch02/failures/narrowing.cpp` 单独触发它。预期的是“列表初始化拒绝窄化转换”这一诊断类别，而不是某一版 GCC 的完整英文措辞。根因是从 `double` 字面量到 `int` 会丢掉小数；修复不是换成圆括号躲开检查，而是先决定业务舍入规则，再显式转换。

立即练习：分别编译 `double price{10};` 与 `int quantity{10.0};`。前者的整数常量 10 能被 `double` 精确表示，可以通过；后者从浮点类型转为整数，即使写成 10.0 也属于列表初始化禁止的窄化，必须被拒绝。再把它改成 10.5，说明同一诊断规则，而不是误记成“没有小数部分就可以”。

2.3 表达式、auto 与显式转换

2.3.1 场景：价格乘数量还不够

名义金额是价格与数量的乘积，平均值还要除以选中记录数。这里会同时遇到运算符优先级、整数除法、结果类型和除零边界。Python 常把数值提升留给运行时；C++ 会先根据操作数类型决定表达式类型。

2.3.2 语法规则与可运行路径

算术运算符包括 `+`、`-`、`*`、`/` 和整数余数 `%`。乘除先于加减；括号既能改变顺序，也能让审阅者看清意图。比较运算 `<`、`<=`、`>`、`>=`、`==`、`!=` 产生 `bool`。

下面的完整程序由构建目标 `ch02_expressions` 编译，并输出三个可独立核算的结果：

Listing 2.2: 算术、类型推导与显式转换

```
1 #include <stdint>
2 #include <iostream>
3
4 int main() {
5     double price{100.5};
6     std::int64_t quantity{10};
7     int selected{2};
8
9     auto notional = price * static_cast<double>(quantity);
10    double average = notional / static_cast<double>(selected);
11    int integer_division = 5 / 2;
```

```

12
13     std::cout << "notional=" << notional << " average=" << average
14         << " integer_division=" << integer_division << '\n';
15 }

```

`auto` 让编译器从初始化表达式推导对象类型；它不是 Python 式动态类型。这里推导结果仍固定为 `double`。必须同时写初始化器，不能写孤立的 `auto notional`；。当类型本身承载业务信息时仍应显式写出，例如数量使用 `std::int64_t`，不要为了少打字全部改成 `auto`。

`static_cast<double>(quantity)` 明确表达“此处要以浮点数参与计算”。对本例的乘法，即使不写转换，`double` 价格也会促使数量转换为 `double`；保留显式转换是为了把边界写给读者，并为随后的除法建立习惯。程序依次得到 `notional=1005`、`average=502.5` 和 `integer_division=2`；最后一个结果来自两个 `int` 的 `5 / 2`。

复合赋值 `buy_notional += notional`；等价于按该运算规则更新左侧对象；`++selected`；把整数加一。二者都会修改左侧对象；如何声明只读对象留到第 3 章。

注Python 迁移提示：Python 的 `/` 总是产生浮点除法，`//` 才是向下取整除法；C++ 的 `/` 取决于两侧类型，两个整数相除先得到整数结果。`auto` 只省略静态类型拼写，不允许对象之后换类型。

2.3.3 失败诊断与立即练习

若平均值意外变成 602，先打印或说明除法两侧的类型，不要先怀疑格式化。另一个常见失败是选中数为零时仍做除法；浮点除零可能得到无穷而继续污染结果，整数除零则是更严重的运行期错误。正确做法是在控制流中显式保护分母。

立即练习：分别预测并运行 `7 / 3`、`7.0 / 3`、`static_cast<double>(7) / 3`。写出实际结果，并指出哪一步发生类型转换。

2.4 条件表达式、if 与 switch

2.4.1 场景：有效买单必须同时通过多道门

本章规则要求证券编号、价格、数量都为正，而且方向必须是买。任何一项失败都拒绝该行。把条件拆清楚比写一个“聪明”的长表达式更适合面试解释和生产审阅。

2.4.2 布尔规则与短路求值

逻辑与写作 `&&`，逻辑或写作 `||`，逻辑非写作 `!`。`a && b` 只有两侧都为真才为真，并且当 `a` 已为假时不会求值 `b`；`a || b` 在 `a` 已为真时不会求值 `b`。这种短路行为既能表达保护条件，也意味着右侧可能根本不执行，不能偷偷依赖它产生副作用。

完整分支实验同时包含布尔表达式、`switch` 和 `if/else`，给定内置行情时输出 `selected`：

Listing 2.3: 离散方向分派与有效性筛选

```

1  #include <cstdint>
2  #include <iostream>
3
4  int main() {
5      std::int64_t symbol_id{101};
6      int side_code{1};
7      double price{100.5};
8      std::int64_t quantity{10};
9

```

```

10  bool is_buy{};
11  switch (side_code) {
12      case 1:
13          is_buy = true;
14          break;
15      case 2:
16          is_buy = false;
17          break;
18      default:
19          is_buy = false;
20          break;
21  }
22
23  bool valid_row = symbol_id > 0 && price > 0.0 && quantity > 0;
24  if (is_buy && valid_row) {
25      std::cout << "selected\n";
26  } else {
27      std::cout << "rejected\n";
28  }
29  }

```

`if` 后的圆括号放条件；条件为真执行第一对花括号，为假执行 `else` 的花括号。即使分支只有一行，本书也保留花括号，避免后来新增语句时出现视觉缩进与真实控制流不一致。

方向代码是少量离散整数，适合用清单中的 `switch` 明确列出分支。`case` 标签必须是可在编译期确定的离散值。`break` 离开当前 `switch`；漏写时会继续执行下一标签，这叫贯穿，除非有明确意图和注释，否则通常是缺陷。`default` 接住未列出的代码，使新出现的脏数据不会悄悄被当成买单。

注Python 迁移提示： Python 的 `if/elif/else` 与 C++ 分支目标相似，但 C++ 用圆括号和花括号界定结构，并要求条件可转换为布尔值。C++ `switch` 不是 Python `match` 的完整对应物：这里只把它当作离散整数分派，不期待结构模式匹配。

2.4.3 失败诊断与立即练习

`if (side_code = 1)` 是赋值，不是比较；对象被改成 1，条件随后为真。高警告级别通常会提示这一可疑写法。修复为 `==`，并保持 `-Wall -Wextra -Wpedantic`。另一个边界是把 `price >= 0.0` 写成有效规则，这会错误接纳零价格；先从业务条件写出真值表，再翻译成表达式。

立即练习：把有效规则改成“买单且价格至少 10，数量在 1 到 100（含端点）”。用恰好位于 10、1、100 的三条记录验证端点，再用价格 9.99 验证拒绝路径。

2.5 for、while 与作用域

2.5.1 场景：已知行数与未知次数是两种循环

输入开头已经给出行数，适合用 `for` 精确处理五次；风险预算能接纳多少笔交易要到条件失败时才知道，适合用 `while`。选择循环形式是在表达终止依据，不只是个人风格。

2.5.2 计数循环与对象可见范围

计数循环也有独立、可构建的最小程序。它检查 0 到 4 的余数，选中三个偶数行：

Listing 2.4: 计数循环与块作用域

```

1  #include <iostream>
2
3  int main() {
4      int selected{};
5      for (int row{}; row < 5; ++row) {
6          int remainder = row % 2;
7          if (remainder == 0) {
8              ++selected;
9          }
10     }
11
12     std::cout << "selected=" << selected << '\n';
13 }

```

圆括号内由两个分号分隔的三个部分依次是：只执行一次的初始化 `int row{};`；每轮开始前检查的条件 `row < 5`；每轮末尾执行的更新 `++row`。`row` 因而依次为 0、1、2、3、4，共五轮；若误写成 `row <= 5` 就会多处理一轮。

每轮先用 `row % 2` 计算余数，并创建本轮的 `remainder`。余数为零时，`if` 分支把循环外的 `selected` 加一，因此第 0、2、4 轮共更新三次并输出 `selected=3`。花括号还建立作用域：`remainder` 每轮重新创建并在该轮末尾销毁，`selected` 则跨轮保存累计值；循环结束后也不能访问初始化部分声明的 `row`。把对象放在“足够用的最小作用域”能减少误用，也让读者更容易追踪状态。

2.5.3 条件循环的完整练习答案

当循环次数由状态决定时，把条件放在 `while` 后。下面的完整答案从 3200 元风险预算中反复扣除每笔 750 元，只有预算足够时才进入下一轮：

Listing 2.5: 用 while 计算可接纳交易数

```

1  #include <iostream>
2
3  int main() {
4      double remaining_budget{3200.0};
5      double risk_per_trade{750.0};
6      int accepted_trades{};
7
8      while (remaining_budget >= risk_per_trade) {
9          remaining_budget -= risk_per_trade;
10         ++accepted_trades;
11     }
12
13     std::cout << "accepted_trades=" << accepted_trades
14               << " remaining_budget=" << remaining_budget << '\n';
15 }

```

初始预算 3200；四轮后剩 200，第五轮开始前 `200 >= 750` 为假，因此接纳数为 4、剩余预算为 200。循环体必须让条件朝终止方向变化；若忘记扣减预算，就会形成无限循环。

注Python 迁移提示：两种语言的 `while` 都在每轮前检查条件。Python 常写 `for row in range(n)`；C++ 的经典 `for` 把初始化、条件和更新显式放在一处。本章还没有容器，所以暂不使用范围 `for`，它会在第 4 章教学。

2.5.4 失败诊断与立即练习

循环结果差一条时，手工列出“进入本轮前的 `row`、条件结果、本轮后的 `row`”三列，比盯着代码更快。程序长时间不结束时，检查每条分支是否都会更新终止状态，以及输入失败后是否还在重复使用旧值。

立即练习：把风险预算改为 2999、每笔风险改为 1000。先预测循环次数与余数，再运行验证，结果应为 `accepted_trades=2 remaining_budget=999`。

2.6 整合：读取、筛选并汇总固定行情

下面是本章输出任务的完整源码。它只组合已经教学的对象、初始化、表达式、转换、分支、循环和作用域。这里补齐输入与错误流规则，不要求读者从旧版第 1 章猜语法：`std::cin >> row_count` 把以空白分隔的文本转换后写入对象，多个 `>>` 可以从左到右连续提取；输入失败时流状态在条件中为假，前缀 `!` 将它取反。`std::cerr` 用于错误消息，正常结果仍送到已经教学的 `std::cout`。

Listing 2.6: 固定行情筛选与汇总

```

1  #include <cstdint>
2  #include <iostream>
3
4  int main() {
5      int row_count{};
6      if (!(std::cin >> row_count) || row_count < 0) {
7          std::cerr << "invalid row count\n";
8          return 1;
9      }
10
11     int selected{};
12     int rejected{};
13     double buy_notional{};
14
15     for (int row{}; row < row_count; ++row) {
16         std::int64_t symbol_id{};
17         int side_code{};
18         double price{};
19         std::int64_t quantity{};
20         if (!(std::cin >> symbol_id >> side_code >> price >> quantity)) {
21             std::cerr << "invalid market row " << row << '\n';
22             return 1;
23         }
24
25         bool is_buy{};
26         switch (side_code) {
27             case 1:
28                 is_buy = true;
29                 break;
30             case 2:
31                 is_buy = false;
32                 break;
33             default:

```

```

34     is_buy = false;
35     break;
36 }
37
38 bool valid_row = symbol_id > 0 && price > 0.0 && quantity > 0;
39 if (is_buy && valid_row) {
40     auto row_notional = price * static_cast<double>(quantity);
41     buy_notional += row_notional;
42     ++selected;
43 } else {
44     ++rejected;
45 }
46 }
47
48 double average{};
49 if (selected > 0) {
50     average = buy_notional / static_cast<double>(selected);
51 }
52
53 std::cout << "selected=" << selected << " rejected=" << rejected
54           << " buy_notional=" << buy_notional << " average=" << average
55           << '\n';
56 }

```

2.6.1 按执行顺序走一遍

1. row_count{} 先得到零；输入成功后变成 5。若读取失败或行数为负，程序向错误流报告并返回非零状态。
2. 三个累计对象从零开始。它们声明在循环外，因此每轮的更新会保留到下一轮。
3. 每轮创建四个字段对象并读取一行。第 1 行方向为买、各字段为正，行名义金额为 $100.5 \times 10 = 1005$ 。
4. 第 2 行是卖，第 3 行数量为零，第 5 行方向代码未知，三者都走拒绝分支。第 4 行贡献 $10.25 \times 3 = 30.75$ 。
5. 循环结束时总额为 $1005 + 30.75 = 1035.75$ ，选中数为 2。分母保护通过后，平均值为 $1035.75/2 = 517.875$ 。
6. 输出流依次写出标签与对象值，所以得到本章开头约定的确定字符串。

在 Linux/WSL 中，从项目根目录运行：

```

cmake -S . -B build
cmake --build build --target ch02_market_filter
./build/ch02_market_filter < code/ch02/data/market_rows.txt
ctest --test-dir build -R ch02_ --output-on-failure

```

Windows 的生成器可能把程序放到配置子目录，或在文件名后加 .exe；源码行为和测试期望不变。CTest 的预期字符串是独立写入的已知结果，不调用被测程序的算法重新计算自己。

2.7 自测、练习与面试表达

先不看答案，逐项给出定义、使用时机、代价或失败例子：

1. 初始化和赋值分别发生在对象生命周期的什么位置？`int x;` 与 `int x{}`；对局部对象有何差异？
2. 为什么 `auto` 不是动态类型？何时宁可显式写 `std::int64_t`？
3. 解释 `5 / 2` 与 `5.0 / 2` 的结果，并给出一个平均值被整数截断的失败例子。

4. `&&` 的短路规则是什么？它怎样保护右侧表达式，又可能怎样隐藏副作用？
5. `switch` 中漏写 `break` 会怎样？为什么仍应保留 `default`？
6. 分别给出适合 `for` 和 `while` 的终止依据，并说明一个无限循环的根因。

编码练习：

1. 从空文件开始重写行情筛选器，不复制清单。先只输出 `selected` 和 `rejected`，再加入总额与平均值；使用给定输入达到本章约定输出。完整可运行答案是 `code/ch02/market_filter.cpp`。
2. 写一个 `while` 程序：风险预算足够时接纳一笔固定风险交易，最后打印接纳数和余数。用 3200 与 750 得到 4 和 200。完整可运行答案是 `code/ch02/exercises/risk_budget_solution.cpp`。
3. 单独编译窄化失败实验，记录失败阶段和诊断类别；然后提出两种修复：改变目标类型，或在明确舍入政策后显式转换。不要把关闭警告当成修复。

问题 2.1 面试检查点 用 90 秒回答：“Python 名称绑定与 C++ 对象初始化有何不同？为什么花括号、显式类型和短路条件对行情入口有价值？”答案至少包含一个可观察输出、一个编译期失败和一个循环边界，而不是只罗列术语。

2.8 本章小结

C++ 程序从具有固定类型和作用域的对象开始；初始化建立初始状态，表达式根据操作数类型产生结果，显式转换标出有意的类型边界。`if` 和 `switch` 选择路径，`for` 和 `while` 重复路径，花括号同时限定控制结构与对象可见范围。你现在已经能只用基础语法完成一个有输入、有脏数据分支、有循环汇总且可测试的量化小程序。后续章节会在这一基础上加入函数、资源生命周期和标准容器。

第三章 函数、引用、const 与多文件组织

内容提要

- ❑ 从重复计算中提取函数，读懂声明、定义、调用、参数和返回值
- ❑ 比较按值、引用和可空指针调用者对象与寿命的可观察影响
- ❑ 用重载表达同一概念的不同参数形式，并辨认候选选择边界
- ❑ 用头文件、实现文件、include guard 和命名空间组织多个翻译单元
- ❑ 诊断缺失定义、重复定义和返回局部对象引用三类故障

注 先修知识：已完成第 1、2 章，能构建程序，并会声明基础类型对象、使用表达式、分支、循环、作用域和标准输入输出；不要求会容器或类。

Python 迁移边界：Python 函数参数仍是名称绑定；C++ 参数声明同时规定类型以及按值复制、引用借用或指针观察，多个源文件还要分别编译再链接。

常见错误与诊断：先确认调用点能否看见声明，再分别检查每个翻译单元是否编译、链接器是否找到唯一定义，以及引用或指针指向的对象是否仍然存活。

3.1 本章任务：把循环中的计算拆成多文件工具

第 2 章的行情筛选器把读取、计算和输出全部写在 `main` 中。程序继续增长时，重复的名义金额公式很容易在不同分支产生不同版本，也无法单独说明哪些函数只读取数据、哪些函数会修改累计状态。本章把计算拆成一个真实的多文件工具：入口从标准输入读取任意多行“价格数量”，实现文件负责累计总名义金额和总数量，最后输出成交量加权平均价（VWAP）。

给定：

```
100.00 10
101.00 20
99.00 10
```

必须逐字符得到：

```
notional=4010.00 quantity=40 vwap=100.25
```

这组结果可独立手算：总数量为 $10 + 20 + 10 = 40$ ，总名义金额为 $100 \times 10 + 101 \times 20 + 99 \times 10 = 4010$ ，所以 VWAP 为 $4010/40 = 100.25$ 。完成判据还包括：项目能从全新构建目录配置；参数实验能预测调用者是否改变；三个故障实验能按编译诊断或链接诊断分类。

3.2 从表达式到函数

3.2.1 场景：同一公式需要一个名字和边界

若每次累计都直接写 `price * quantity`，公式的含义、输入和结果只能靠上下文猜。函数把一段计算命名，并建立局部作用域。最小规则可以读作：

```
return_type function_name(parameter_declarations);
```

这一行是声明：分号表示这里只有函数接口，没有函数体。调用点只要先看见声明，就能检查实参数量与类型。定义重复相同的返回类型、名称和参数列表，随后用花括号给出函数体；定义末尾不再加分号。调用写作函数名、圆括号和实参，例如 `trade_notional(price, quantity)`。

3.2.2 重构路线：一次只移动一个边界

重构从第 2 章循环内的 `total_notional += price * quantity;` 开始，而不是直接重写最终项目。第一步把清单 3.2 的 `trade_notional` 定义放在单文件 `main` 之前，只把右侧表达式替换为函数调用，输出必须不变。第二步把声明放在 `main` 前、把同一定义移到 `main` 后，证明调用点只需先看见声明。第三步才把声明移入清单 3.1 的头文件、定义移入实现文件；最后让 CMake 同时编译入口和实现。每一步都用章首输入复跑，任何输出变化都说明移动边界时顺带改变了行为。

本章接口的真实头文件如下；其余多文件结构稍后逐项解释：

Listing 3.1: 行情计算函数声明与 include guard

```
1 #ifndef QUANT_CH03_MARKET_MATH_HPP
2 #define QUANT_CH03_MARKET_MATH_HPP
3
4 namespace quant::chapter3 {
5
6 double trade_notional(double price, int quantity);
7
8 double trade_notional(double price, int quantity, double multiplier);
9
10 void add_trade(double price, int quantity, double& total_notional,
11               int& total_quantity);
12
13 double volume_weighted_price(double total_notional, int total_quantity);
14
15 } // namespace quant::chapter3
16
17 #endif
```

3.2.3 定义、参数、返回与局部作用域

实现文件给出这些声明的定义：

Listing 3.2: 行情计算函数定义

```
1 #include "quant/ch03/market_math.hpp"
2
3 namespace quant::chapter3 {
4
5 double trade_notional(double price, int quantity) {
6     return price * quantity;
7 }
8
9 double trade_notional(double price, int quantity, double multiplier) {
10     return trade_notional(price, quantity) * multiplier;
11 }
```

```

12
13 void add_trade(double price, int quantity, double& total_notional,
14               int& total_quantity) {
15     total_notional += trade_notional(price, quantity);
16     total_quantity += quantity;
17 }
18
19 double volume_weighted_price(double total_notional, int total_quantity) {
20     if (total_quantity == 0) {
21         return 0.0;
22     }
23     return total_notional / total_quantity;
24 }
25
26 } // namespace quant::chapter3

```

以第一个函数为例，`double` 是返回类型；圆括号内依次声明两个参数对象。调用发生时，`price` 和 `quantity` 在函数作用域中创建，`return price * quantity;` 先计算表达式，再把结果交回调调用点，同时结束本次调用。两个参数以及函数体内创建的其他局部对象都在离开右花括号时结束生命周期。

`volume_weighted_price` 先检查分母。`return 0.0;` 是早返回：条件成立时函数立即结束，不再执行后面的除法。这里用零表示“没有成交量”的简化教学协议；生产接口往往需要显式表达“无结果”，第 10 章再比较可选值和错误机制。

声明中的参数名可以省略，但教学与审阅代码保留有意义的名称。定义必须让每个名称对应函数体中的对象。返回类型不是注释：声明承诺返回 `double`，调用表达式也因此具有 `double` 类型。

注Python 迁移提示：两种语言都用函数名和圆括号调用，也都有局部作用域与早返回。Python 在运行时把实参对象绑定给参数名称；这里的 C++ 声明在编译时固定参数和返回类型，并决定后续是复制还是引用。Python 函数可在不同路径返回不同种类对象，C++ 每条返回路径必须满足同一声明的返回类型。

失败诊断：若调用写在声明之前，编译器会报告名称未声明；若声明写作 `double trade_notional(double, int);`，定义却交换参数类型，二者就是不同函数，原调用可能在链接阶段找不到定义。修复时同时对照声明、定义和调用，不要只改其中一处。

立即练习：只把 `volume_weighted_price` 的零数量返回值从 0.0 改为 -1.0，用空输入运行工具，结果中的 VWAP 应从 0.00 变为 -1.00；随后恢复原协议。

3.3 按值、引用与可空指针

3.3.1 场景：函数是否可以改动调用者

接口不仅说明“接收一个 `double`”，还要说明函数获得独立副本还是借用调用者对象。下面的完整实验由 CTest 逐字符检查：

Listing 3.3: 三种参数模式的可观察差异

```

1 #include <iostream>
2
3 double add_tick(double price) {
4     price += 1.0;
5     return price;
6 }

```

```

7
8 double observe_price(const double& price) {
9     return price;
10 }
11
12 void charge_fee(double& cash, double fee) {
13     cash -= fee;
14 }
15
16 int main() {
17     const double original{100.0};
18     const double copied_result{add_tick(original)};
19     const double observed{observe_price(original)};
20
21     double cash{1000.0};
22     charge_fee(cash, 5.0);
23
24     const double* price_view{&original};
25     const double pointed{*price_view};
26     const double* missing{nullptr};
27
28     std::cout << "original=" << original << " copied_result=" << copied_result
29               << " observed=" << observed << " cash=" << cash
30               << " pointed=" << pointed << " null=" << (missing == nullptr)
31               << '\n';
32     return 0;
33 }

```

`double price` 是按值参数。`add_tick` 修改自己的参数对象并返回 101，但调用者的 `original` 仍为 100。对基础数值类型，按值通常最直接。

`const double& price` 是只读左值引用。`&` 表示参数引用调用者已有对象，不创建独立的业务值；前面的 `const` 禁止函数通过该名称赋值。`observe_price` 因而能读取 `original`，却不能写 `price += 1.0`；。对一个 `double`，只读引用通常没有性能优势；这里用小类型把语义显示清楚，后续较大的标准库和领域对象才更常用 `const T&`。

`double& cash` 是可变左值引用。`charge_fee` 对 `cash` 的赋值直接改变调用者对象，所以输出为 995。可变引用适合“函数职责就是修改这个对象”的接口；若函数主要产生一个新值，返回值通常更易组合和测试。调用语法不会额外显示 `&`，因此名称和文档应让副作用容易发现。

`const double original{100.0}`；还演示只读对象：初始化之后不能再赋值。`const` 约束的是通过该名称允许的操作，不表示数值被放到某种神秘的永久存储中。

注Python 迁移提示： Python 的实参传递不是“所有东西按引用”：参数名称绑定到同一对象，但对名称重新赋值不会改调用者，而修改共享的可变对象可能会。C++ 把三种意图写进函数签名：按值得到独立对象，`const&` 只读借用，`&` 可变借用。二者不能只靠表面调用语法互相类推。

失败诊断：把字面量 100.0 传给 `double&` 会在编译阶段失败，因为可变左值引用需要一个可修改、寿命明确的调用者对象；改成 `double cash{100.0}`；`charge_fee(cash, 5.0)`；。若只读函数意外要求可变引用，`const` 对象也无法调用它；正确修复是收窄函数权限为 `const double&` 或按值，而不是删除调用者的 `const`。

立即练习：只把 `charge_fee` 的第一个参数改成按值，保持函数体和调用点不变。预测局部参数仍会减去 5，但最终 `cash` 应从 995 变回 1000；运行验证后恢复可变引用。

3.3.2 指针：允许“没有观察对象”

引用必须绑定对象且不能改绑；指针对象保存地址，也可以保存空指针值。清单 3.3 中的三行摘录为：

```
1 const double* price_view{&original};
2 const double pointed{*price_view};
3 const double* missing{nullptr};
```

第一行用取地址运算符 & 得到 original 的地址；类型中的 * 声明“指向只读 double 的指针”。第二行的 *price_view 解引用地址并读取 100。相同符号依上下文承担声明符或运算符职责，不能只按字符背诵。

const double* missing{nullptr}; 明确表示当前没有对象；程序用 missing == nullptr 得到 1。指针可以为空，所以每次解引用前都必须由接口契约或条件证明非空。这里的裸指针只是非拥有观察，不负责销毁对象；price_view 也不能比 original 活得更久。所有权与数组指针在第 6 章继续展开。

注Python 迁移提示：Python 常用 None 表示没有对象；C++ 裸指针用 nullptr 表示没有地址。但 Python 名称会自动保持对象引用，非拥有 C++ 指针不会延长被观察对象的生命周期。引用适合必有对象，指针适合“可能没有”的观察接口。

失败诊断：直接计算 *missing 是未定义行为，不能把某次崩溃当成契约；修复是在解引用前检查非空，并让被指向对象覆盖整个使用期。立即练习：只把 price_view 初始化为 nullptr，用 if (price_view != nullptr) 保护读取；预期程序跳过读取而不崩溃，随后恢复指向 original。

3.4 重载：同名操作的不同参数表

同一领域动作有时存在不同的输入形式。本章头文件声明了两个 trade_notional：第一个接收价格和数量；第二个额外接收乘数。它们名称相同，参数数量不同，这叫函数重载。调用 trade_notional(price, quantity) 只有两个实参，编译器选择二参数候选；三实参调用选择另一个定义。三参数版本复用二参数版本，再乘以 multiplier，避免复制基础公式。

重载的签名必须在参数数量或参数类型上可区分；只改变返回类型不够，因为调用点不能总从接收结果的上下文唯一决定候选。默认实参也可能让两个候选同时可用，从而产生“调用有歧义”类别的编译错误。本书优先让重载表达同一概念；完全不同的业务动作应使用不同名称。

注Python 迁移提示：Python 后定义的同名函数通常替换前一个绑定，常用默认参数或 *args 手工分派。C++ 会在编译期建立同名候选集，根据实参数量、类型和所需转换选择唯一最佳匹配；没有可行候选或最佳候选不唯一都会编译失败。

失败诊断：新增 double trade_notional(double price, int quantity, double multiplier = 1.0); 后，二实参调用可能同时匹配原二参数版本和带默认实参的版本。根因是接口集合有歧义；修复为移除默认值、移除重复能力，或使用含义不同的名称。

立即练习：只把 add_trade 内部的二实参调用改为 trade_notional(price, quantity, 1.0)。结果应逐字符不变，因为乘数为 1；然后把乘数改为 0.5，预期总名义金额与 VWAP 都减半，而总数量仍为 40。

3.5 头文件、实现文件与翻译单元

3.5.1 场景：让不同源文件共享同一接口

每个 .cpp 会先经过预处理，再独立编译成一个翻译单元的目标文件。入口 main.cpp 需要看见函数声明，实现文件 market_math.cpp 负责给出定义。最后，链接器把两个目标文件组合成程序。头文件是共享声明的位置，不是“自动参与构建”的魔法文件。

#include "quant/ch03/market_math.hpp" 使用双引号查找项目头文件；标准库头文件继续使用尖括号。预处理器把头文件内容带入当前翻译单元。头文件可能经不同路径被重复包含，因此清单使用 include guard：

`#ifndef` 检查宏尚未定义, `#define` 建立标记, 末尾 `#endif` 关闭条件区域。同一翻译单元第二次读到它时, 声明区域会被跳过。

`namespace quant::chapter3 { ... }` 把名称放进项目自己的命名空间, 降低与其他库同名函数冲突的风险。调用点写全限定名 `quant::chapter3::add_trade`; 头文件不写 `using namespace`, 避免把选择强加给每个包含者。

`include guard` 立即练习: 只在入口中连续写两次同一个项目头文件, 然后运行预处理命令 `g++ -E -P -I code/ch03/include code/ch03/src/main.cpp`。输出中两个 `trade_notional` 重载声明应各出现一次, 而不是各出现两次; 这就是 `guard` 跳过第二次展开的可观察证据。删除重复的 `include` 后再继续构建。

入口文件只负责读取、调用和输出:

Listing 3.4: 多文件行情工具入口

```

1  #include "quant/ch03/market_math.hpp"
2
3  #include <iomanip>
4  #include <iostream>
5
6  int main() {
7      double total_notional{};
8      int total_quantity{};
9
10     double price{};
11     int quantity{};
12     while (std::cin >> price >> quantity) {
13         quant::chapter3::add_trade(price, quantity, total_notional,
14                                     total_quantity);
15     }
16
17     if (!std::cin.eof()) {
18         std::cerr << "input-error: expected price and quantity\n";
19         return 2;
20     }
21
22     const double vwap{
23         quant::chapter3::volume_weighted_price(total_notional, total_quantity)};
24     std::cout << std::fixed << std::setprecision(2)
25               << "notional=" << total_notional << " quantity=" << total_quantity
26               << " vwap=" << vwap << '\n';
27     return 0;
28 }
```

`std::cin >> price >> quantity` 每轮创建一组输入值; `add_trade` 通过两个可变引用更新循环外的累计对象。输入因文件结束而停止是正常路径; 若因为无法转换的文本停止, 程序向 `std::cerr` 写出稳定类别并返回 2。 `const double vwap{...}` 表示结果初始化后只读。输出用 `std::fixed` 和 `std::setprecision(2)` 固定两位小数; 它们改变后续流格式, 不改变内部 `double` 值。

独立快照的 `code/ch03/CMakeLists.txt` 同时列出两个 `.cpp`, 并用 `target_include_directories` 把 `include/` 加入该目标的头文件搜索路径。头文件本身不需要列为编译源; 真正分别产生目标文件的是两个实现源文件。

注Python 迁移提示: Python 的 `import` 通常加载模块对象并在运行时绑定名称; C++ 的 `#include` 是预处理期

文本包含，每个 `.cpp` 仍独立编译。头文件提供共同声明，定义则必须在最终链接中恰好满足需要。把 `include guard` 理解成模块加载缓存会得到错误心智模型。

失败诊断：若编译器报告找不到头文件，先检查 `include` 拼写和目标的 `include` 目录；若只有 `main.cpp` 进入目标，编译能通过但链接器会找不到 `market_math.cpp` 中的定义。修复是让正确实现文件参与同一目标，而不是把 `.cpp` 直接 `include` 进入入口文件。

立即练习：只从 `add_executable` 中删除 `src/market_math.cpp`，从全新构建目录编译。预期两个源文件不再同时参与目标，链接阶段报告缺失定义；恢复该文件后重新干净构建，输出应回到章首结果。

3.6 三个诊断实验：定义数量与对象寿命

故意失败文件位于 `code/ch03/failures/`，不进入默认绿色目标。测试只匹配稳定诊断类别，不绑定完整英文文本。

3.6.1 缺失定义

`missing_definition.cpp` 声明并调用 `fee_rate`，却没有函数体。源文件能编译成目标文件，链接时报告 `undefined reference` 或 `unresolved external symbol` 类别。根因不是“头文件没被执行”，而是最终链接集合没有定义。修复是提供签名一致的唯一定义并让对应目标文件参与链接。

3.6.2 重复定义

`duplicate_definition_a.cpp` 与 `duplicate_definition_b.cpp` 各自都能编译，却都定义了同一个 `fee_rate()`。把两个目标文件一起链接时应报告 `multiple definition` 或 `already defined` 类别。这是“一份声明可以被许多翻译单元看见，但非内联普通函数在程序中需要一个定义”的直接证据。修复是把声明留在头文件、定义只留在一个实现文件。

3.6.3 悬空引用

`dangling_reference.cpp` 返回局部 `double` 对象的引用。局部对象在函数返回时已经结束生命周期，调用者拿到的引用随即悬空。高警告级别编译应报告“返回局部对象引用”类别；本实验不运行生成的程序，因为读取悬空引用是未定义行为，不能把某次崩溃或数值当成稳定结果。修复是按值返回这个小对象，或引用一个已证明比调用者活得更久的对象。

立即练习：为三条实验各记录“两个编译是否成功、链接是否成功、是否允许运行”。答案依次是：单一源编译成功但链接失败；两个源分别编译成功但合并链接失败；编译产生寿命警告且禁止把运行结果当证据。

3.7 从干净目录复现与章节验收

在 Linux/WSL 中运行：

```
cmake -S code/ch03 -B build/ch03
cmake --build build/ch03 --target ch03_market_tool
./build/ch03/ch03_market_tool < code/ch03/data/market_rows.txt
cmake -S . -B build-mingw -G "MinGW Makefiles"
cmake --build build-mingw --target ch03_market_tool
ctest --test-dir build-mingw -R ch03_ --output-on-failure
```

前三条命令证明章节快照不依赖后续项目源码并得到精确输出；后三条从仓库根项目建立测试构建树，再运行全部第 3 章证据。Windows 多配置生成器可能把可执行文件放在 `build\ch03\Debug` 并加 `.exe`，但源文件边界、声明/定义规则与测试预期不变。

3.8 自测、编码任务与面试表达

1. 函数声明、定义和调用分别提供什么信息？分号与花括号出现在何处？
2. 参数对象和返回值的生命周期边界是什么？早返回怎样改变控制流？
3. 按值、`const T&`、`T&` 与 `const T*` 分别允许观察或修改什么？哪个可以为 `nullptr`？
4. 重载候选靠什么区分？为什么只改变返回类型不能形成可用重载？
5. `.cpp`、头文件、翻译单元、目标文件和可执行文件怎样连接？
6. `include guard` 防止什么？为什么项目头文件通常使用双引号和命名空间？
7. 缺失定义、重复定义和悬空引用分别在哪个阶段得到什么证据？

编码任务：

1. 从空目录重建 `code/ch03` 的头文件、实现文件和入口，使用固定输入得到章首精确输出，并从全新构建树复现。
2. 为行情工具增加只读函数 `double basis_points(const double& rate)`，返回 `rate * 10000.0`；用 0.0015 验收结果 15，并证明调用者值不变。完整答案见 `code/ch03/exercises/basis_points_solution.cpp`。
3. 运行三个故障实验，记录每个翻译单元的编译状态、链接状态、稳定类别、根因和最小恢复步骤；不要运行悬空引用程序来“看看会怎样”。

问题 3.1 面试检查点 用 90 秒回答：“如何把一个单文件行情程序拆成可链接的多文件 C++ 程序？”回答需包含声明与定义、按值和引用、`include guard`、命名空间、独立翻译单元以及一个缺失或重复定义的可观察诊断。

3.9 本章小结

函数声明让调用点在编译期检查接口，定义提供唯一实现，调用创建参数对象并取得返回结果。按值表达独立副本，`const&` 表达只读借用，`&` 表达可变借用，指针还可用 `nullptr` 表示没有观察对象；重载让同名动作拥有可区分的参数表。头文件共享声明，`include guard` 控制重复包含，命名空间管理名称，多个 `.cpp` 分别编译后由链接器组合。诊断时先问每个翻译单元是否编译，再问程序是否拥有所需且唯一的定义，最后确认引用或指针仍指向存活对象。

第四章 STL、迭代器、算法与 ranges

内容提要

- 根据访问模式选择连续、节点式和关联容器
- 使用 ranges 构造惰性、可组合的数据视图
- 用迭代器理解算法与数据表示的解耦
- 识别复杂度、分配与迭代器失效成本

注 先修知识：会使用 Python 列表、字典、生成器与推导式。

Python 迁移边界：ranges 视图类似惰性生成器，但受静态类型、借用范围和迭代器失效约束。

常见错误与诊断：结果异常时先检查视图底层对象寿命、迭代器失效、谓词副作用和浮点归约顺序。

4.1 先从 vector 开始

`std::vector<T>` 的连续存储对缓存和预取友好，通常应成为序列容器默认选择。只有测量和操作需求证明频繁中间插入、稳定地址或双端增长重要时，才考虑其他结构。链表虽然单次已定位插入是常数时间，但遍历中的指针追逐和分配成本常使它输给移动一段连续内存。

`reserve` 预留容量但不改变元素数量；`resize` 改变大小并构造元素。扩容可能使所有指针、引用和迭代器失效，这是常见生命周期错误来源。

4.2 算法表达意图

`std::sort`、`std::transform`、`std::accumulate` 和查找算法比手写循环更直接表达操作类别，也让复杂度预期清楚。算法不是为了消灭所有循环；涉及多状态推进、早停或需要精细诊断的业务逻辑，命名良好的普通循环可能更可读。

迭代器把“元素位置”抽象为协议。随机访问算法要求更强迭代器能力，`concepts` 会在编译期拒绝不满足要求的范围。不要默认每个迭代器都支持加整数或常数时间距离。

4.3 ranges 是视图，不是新容器

视图通常不拥有元素，而是惰性描述如何遍历底层范围：

Listing 4.1: 筛选买入成交并聚合名义金额

```
1 #include <iostream>
2 #include <ranges>
3 #include <vector>
4
5 #include "quant/types.hpp"
6
7 int main() {
8     const std::vector<quant::Fill> fills{
9         {{}, "AAPL", quant::Side::buy, 10, 187.5},
10        {{}, "AAPL", quant::Side::sell, 4, 188.0},
11        {{}, "MSFT", quant::Side::buy, 5, 420.0},
12        {{}, "MSFT", quant::Side::sell, 2, 421.0},
13    };
14 }
```

```

14
15     auto buys = fills | std::views::filter([](const quant::Fill& fill) {
16         return fill.side == quant::Side::buy;
17     });
18
19     double positive_notional = 0.0;
20     for (const quant::Fill& fill : buys) {
21         positive_notional += fill.price * static_cast<double>(fill.quantity);
22     }
23     std::cout << "positive-notional=" << positive_notional << '\n';
24 }

```

`filter` 不立即复制买入成交；循环拉取时才执行谓词。优点是组合和避免中间容器，代价是视图生命周期依赖底层范围，重复遍历可能重复计算。视图是否更快必须结合访问次数和编译结果测量。

注Python 迁移提示：生成器表达式与 `ranges` 视图都具有惰性，但 C++ 视图的类型编码了适配器组合，并受到借用范围和迭代器失效规则约束。把临时容器接入长期保存的视图前，必须确认所有权。

4.4 关联查找与散列

`std::map` 提供有序树结构和对数复杂度，`std::unordered_map` 提供平均常数查找但有散列、桶和重哈希成本。持仓按代码查找常可从后者开始；若需要有序遍历、范围查询、稳定最坏界或确定性输出，则树结构可能更合适。

小数据集上，排序向量加二分查找经常优于节点容器，因为连续布局减少间接访问。选择数据结构应同时考虑规模、更新频率、遍历模式和延迟尾部，而不是只背复杂度表。

4.5 数值归约的注意事项

浮点加法不满足结合律，并行归约或改变遍历顺序可能改变末位结果。绩效统计应规定允许误差、稳定算法和可重复性要求。对于金额，先决定表示和舍入规则；对于收益与方差，考虑补偿求和或在线稳定算法，而不是盲目累加。

4.6 常见错误

1. 在遍历 `vector` 时插入并继续使用旧迭代器。
2. 为一次线性扫描建立昂贵的散列表。
3. 返回引用临时容器的视图。
4. 在谓词中加入不可见副作用，使算法结果依赖调用次数。
5. 只比较理论复杂度，不测量真实规模下的缓存和分配成本。

问题 4.1 面试检查点 为什么 `vector` 常比 `list` 快？`reserve` 与 `resize` 有什么区别？`ranges` 视图拥有什么、不拥有什么？何时有序向量可能胜过哈希表？

4.7 练习

1. 用 `ranges` 筛选指定代码的买入成交，并计算加权平均价格。
2. 对比 `vector` 线性扫描、排序后二分查找和 `unordered_map` 在不同持仓数量下的成本。

3. 构造一次扩容导致引用失效的最小例子，并提出两种修复。
4. 解释并行浮点归约为何可能不逐位可重复，定义合理测试断言。

4.8 本章小结

STL 的核心不是容器清单，而是表示、迭代协议和算法复杂度的组合。连续存储是强默认，`ranges` 提供非拥有惰性视图，任何性能判断都必须回到真实数据规模和访问模式。

第二部分

写出可靠组件

第五章 类、接口与值语义：设计量化组件

内容提要

- ❑ 从不变量和用例出发设计类，而不是从字段出发
- ❑ 把撮合与组合状态隔离为可替换组件
- ❑ 用值语义表达事件、订单与成交
- ❑ 避免贫血模型、万能管理器和继承滥用

注 先修知识：理解类、组合、接口和前四章的值语义。

Python 迁移边界：Python 对象默认通过引用共享；本章优先让领域事实成为独立值，并用类维护复合不变量。

常见错误与诊断：若调用者必须按特定顺序调用多个 `setter`，说明边界过浅；从不变量重新设计单一操作。

5.1 类的价值是守住边界

类不应只是把公开字段换成机械 `getter/setter`。一个有用的类拥有清晰责任、维护不变量，并通过小型公开接口隐藏内部表示。组合模块的责任是：成交发生后，现金与持仓必须一起改变；调用者不应分别调用 `set_cash` 和 `set_position`。

本书首先把事件、订单和成交设计为值类型。它们代表某个时刻的事实或意图，复制后仍然具有同样含义，不需要共享身份：

Listing 5.1: 回测项目的基础值类型

```
1 #pragma once
2
3 #include <chrono>
4 #include <cstdint>
5 #include <string>
6
7 namespace quant {
8
9     using Timestamp = std::chrono::sys_time<std::chrono::milliseconds>;
10
11     enum class Side : std::uint8_t { buy, sell };
12
13     struct MarketEvent final {
14         Timestamp timestamp{};
15         std::string symbol;
16         double price{};
17         std::int64_t quantity{};
18     };
19
20     struct OrderIntent final {
21         std::string symbol;
22         Side side{Side::buy};
23         std::int64_t quantity{};
24     };
25 }
```



```

26 struct Fill final {
27     Timestamp timestamp{};
28     std::string symbol;
29     Side side{Side::buy};
30     std::int64_t quantity{};
31     double price{};
32 };
33
34 struct PortfolioSnapshot final {
35     std::string symbol;
36     std::int64_t quantity{};
37     double cash{};
38     double equity{};

```

值语义让容器、测试和消息传递更直接。若以后测量证明字符串复制是热点，可以引入代码表、驻留字符串或列式布局；在证据出现前，清楚所有权比提前压缩表示更重要。

5.2 组合优于继承

回溯引擎拥有撮合器和组合对象，它不需要继承二者。继承适用于真实的可替换子类型关系，并要求基类接口对所有派生类都成立。为了复用几行实现而建立继承层次，往往产生受保护状态、脆弱覆盖和难以理解的生命周期。

运行时多态可以用纯虚接口或类型擦除；编译期多态可以用模板与 concepts。选择依据是替换发生在编译期还是运行时、ABI 是否稳定、代码体积和诊断是否可接受，而不是“现代 C++ 永远不用虚函数”。

5.3 撮合与组合的公开契约

Listing 5.2: 撮合和组合的最小公开接口

```

1 #pragma once
2
3 #include <cstdint>
4 #include <optional>
5 #include <string>
6 #include <unordered_map>
7
8 #include "quant/types.hpp"
9
10 namespace quant {
11
12 class SimulatedExchange final {
13 public:
14     [[nodiscard]] std::optional<Fill> match(const OrderIntent& order,
15                                           const MarketEvent& event) const {
16         if (order.symbol != event.symbol || order.quantity <= 0 ||
17             event.quantity < order.quantity || event.price <= 0.0) {
18             return std::nullopt;
19         }

```

```

20     return Fill{event.timestamp, order.symbol, order.side, order.quantity,
21                 event.price};
22 }
23 };
24
25 class Portfolio final {
26 public:
27     explicit Portfolio(double initial_cash) : cash_(initial_cash) {}
28
29     void apply_fill(const Fill& fill) {
30         const auto delta = signed_quantity(fill.side, fill.quantity);
31         positions_[fill.symbol] += delta;
32         cash_ -= static_cast<double>(delta) * fill.price;
33     }
34
35     [[nodiscard]] PortfolioSnapshot snapshot(const std::string& symbol,
36                                             double mark_price) const {
37         const auto found = positions_.find(symbol);
38         const std::int64_t quantity =
39             found == positions_.end() ? 0 : found->second;
40         return PortfolioSnapshot{symbol, quantity, cash_,
41                                 cash_ + static_cast<double>(quantity) * mark_price};
42     }
43
44 private:
45     double cash_;
46     std::unordered_map<std::string, std::int64_t> positions_;
47 };
48
49 } // namespace quant

```

撮合器拒绝代码不一致、非正数量、流动性不足或非法价格的订单。组合只接受已经形成的成交，并通过快照暴露可观察状态。测试不读取内部散列表，因而未来可以把持仓替换为排序向量或专用簿记结构而不改行为测试。

当前简化组合的权益只覆盖请求快照的单一代码。这是教学边界，不是暗藏假设：第 13 章会明确其适用范围和扩展到多资产估值所需的价格映射、币种与缺失行情政策。

5.4 构造后即有效

理想对象在构造完成后就满足不变量。若配置可能失败，可使用工厂返回值或错误，而不是创建“半有效对象”再要求调用 `init()`。对于整数数量，规定正数与方向分离；对于价格，拒绝非有限值；对于初始现金，生产接口还应规定是否允许负数和币种。

5.5 接口深度

深接口用少量调用隐藏大量正确性工作。例如 `apply_fill` 隐藏买卖方向、现金符号和持仓更新的一致提交。浅接口只是改名转发，迫使调用者理解内部步骤。代码审查时可以问：调用者要读多少实现，才能安全使用

这个类？

问题 5.1 面试检查点 为什么事件适合值语义？组合为何只接收成交而不接收订单？何时运行时多态优于模板？“构造后即有效”如何改变错误处理接口？

5.6 练习

1. 为订单增加唯一标识，决定它属于值的一部分还是外部追踪元数据。
2. 扩展撮合器支持部分成交，写出剩余数量与事件流动性的契约。
3. 设计多资产组合快照接口，说明缺失最新价格时选择拒绝、沿用旧价还是标记结果不完整。
4. 找出一个“Manager”类，把它拆成两个各自拥有明确不变量的组件。

5.7 本章小结

可靠类从职责和不变量开始。事件、订单和成交采用值语义，撮合与组合通过小接口协作，内部数据结构保持可替换。下一章用模板与 `concepts` 把接口要求变成编译期契约。

第六章 对象生命周期、RAII 与所有权

内容提要

- ❑ 从构造、使用到析构，画出自动对象与动态对象的生命周期
- ❑ 区分值、引用、裸指针，以及左值、右值和移动后的有效状态
- ❑ 识读 C 数组、new/delete 和典型悬空风险，同时坚持现代默认路径
- ❑ 用 RAII 和 Rule of Zero 把文件等资源的释放绑定到对象生命周期
- ❑ 在直接值与三种智能指针之间做有理由的所有权选择
- ❑ 用 AddressSanitizer 取得一次真实的 heap-use-after-free 诊断

注 先修知识：已完成基础篇，能读写函数、引用、const、string/vector、struct/class 和构造函数；关键语法会在首次实验处复习。

Python 迁移边界：Python 名称和垃圾回收不等于 C++ 所有权；最接近 RAII 的 Python 心智模型是显式的 with 边界，而不是等待 __del__。

常见错误与诊断：出现悬空、重复释放或泄漏时，先画“谁创建、谁拥有、谁借用、何时销毁”，再用 sanitizer 取得失败类别。

6.1 本章输出：让所有权状态可以被观察

低延迟系统里，“这个指针应该还活着”不是可靠论证。行情快照、文件句柄、锁和异步任务都要求同一个问题有明确答案：对象从何时存在，到何时销毁，谁有权延长它的寿命？本章最终任务是重写一个所有权选择程序，并得到如下精确输出：

```
direct=direct copied=copy unique_empty=1 unique_owner=unique shared_owners=1 weak_expired=0
weak_after_reset=1
```

它依次证明五件事：直接值的复制互相独立；唯一所有权可以转移；移动后原智能指针为空；弱观察不增加共享计数；最后一个共享所有者释放后，弱观察变为失效。完成判据不是背出四种工具，而是能从业务寿命关系推出选择，并用输出验证状态转换。

6.2 对象生命周期、复制与移动

6.2.1 场景：一次行情缓冲区究竟销毁几次

函数内的自动对象通常在声明处开始生命周期，在离开其花括号作用域时结束。构造成功才意味着对象存在；析构完成后，其存储即使尚未被别处覆盖，也不能继续作为原对象使用。多个自动对象按构造的逆序析构，这让后创建的资源可以先释放。

下面两种初始化分别请求复制构造和移动构造：

```
Trace copied{original};
Trace moved{std::move(original)};
```

前者创建独立对象。std::move 本身不搬运字节，也不销毁对象；它把表达式转换为允许选择移动操作的值类别。移动后的源对象仍必须可析构、可赋值，但除非类型另有保证，其具体值通常未指定。

6.2.2 完整实验与逐行轨迹

下面的程序显式记录构造、复制、移动和析构。它是教学探针，不是业务类型应照抄的设计；真实业务类型优先让成员自己管理资源。

Listing 6.1: 复制、移动与逆序析构轨迹

```

1  #include <iostream>
2  #include <string>
3  #include <utility>
4
5  class Trace final {
6  public:
7      explicit Trace(std::string label) : label_(std::move(label)) {
8          std::cout << "construct=" << label_ << '\n';
9      }
10
11     Trace(const Trace& other) : label_(other.label_ + "-copy") {
12         std::cout << "copy=" << other.label_ << "->" << label_ << '\n';
13     }
14
15     Trace(Trace&& other) noexcept : label_(std::move(other.label_)) {
16         other.label_ = "moved-from";
17         std::cout << "move=" << label_ << '\n';
18     }
19
20     ~Trace() { std::cout << "destroy=" << label_ << '\n'; }
21
22     [[nodiscard]] const std::string& label() const { return label_; }
23
24 private:
25     std::string label_;
26 };
27
28 int main() {
29     Trace original{"quote"};
30     {
31         Trace copied{original};
32         Trace moved{std::move(original)};
33         std::cout << "states=" << original.label() << ',' << copied.label() << ','
34             << moved.label() << '\n';
35     }
36     std::cout << "after-inner=" << original.label() << '\n';
37 }

```

`original` 先以标签 `quote` 构造。内部作用域复制出 `quote-copy`，随后把 `original` 的字符串资源移动给 `moved`；本实验主动把源标签设为 `moved-from`，所以这一状态可测试。离开内部作用域时，`moved` 后构造、先析构，然后才是 `copied`。最外层结束时，源对象仍能安全析构。

够用的值类别工作模型是：左值表达式有可重复识别的身份，例如已命名对象；纯右值常表示临时计算结果；将亡值表示资源可以被转移的对象。值类别属于表达式，不是“对象永久贴着的标签”。`original` 是对象，

而写下它这个表达式时通常是左值：`std::move(original)` 才得到将亡值表达式。

注Python 迁移提示：Python 的 `b = a` 通常让两个名称指向同一对象；C++ 的 `Trace b{a}` 调用复制构造并产生独立对象。Python 对象何时回收不应依赖某个实现的引用计数时机；C++ 自动对象离开作用域时按语言规则确定性析构。

6.2.3 失败诊断与立即练习

常见错误是把“移动后仍有效”误读成“移动后仍保持原值”。标准容器通常保证移动后可析构和可重新赋值，但不保证仍有原内容。诊断时先看类型契约，再检查是否在移动后读取了业务状态。

立即练习：把内部两次声明交换顺序，先预测构造和析构行的顺序，再运行 `ch06_lifetime_trace`。只改这一处，并解释为什么析构顺序随构造顺序反转。

6.3 RAII：把释放变成结构性结果

6.3.1 场景：提前返回也必须关闭报告文件

文件、锁、socket、线程和临时目录都需要成对操作。若每条控制路径都手写关闭或解锁，遗漏概率会随分支增加。RAII（Resource Acquisition Is Initialization）让资源由对象成员持有，并在对象析构时释放；作用域和栈展开替程序选择正确清理路径。

6.3.2 完整实验：文件流离开作用域后立即可读

Listing 6.2: 文件流的确定性关闭

```

1  #include <cstdio>
2  #include <fstream>
3  #include <iostream>
4  #include <string>
5
6  int main() {
7      std::string path{"ch06_raii_report.txt"};
8      {
9          std::ofstream output{path};
10         output << "symbol,quantity\nAAPL,10\nMSFT,20\n";
11         std::cout << "inside_open=" << static_cast<bool>(output) << '\n';
12     }
13
14     int rows{};
15     {
16         std::ifstream input{path};
17         std::string line;
18         std::getline(input, line);
19         while (std::getline(input, line)) {
20             ++rows;
21         }
22     }
23
24     int removed = std::remove(path.c_str()) == 0;

```

```

25     std::cout << "after_scope_rows=" << rows << " removed=" << removed << '\n';
26 }

```

内部第一对花括号创建 `std::ofstream`。离开作用域时，它的析构函数刷新并关闭文件；下一作用域因此能够立即打开并读取两行数据。输入流也先离开作用域，Windows 才允许后面的 `std::remove` 删除文件。测试得到 `inside_open=1`、`after_scope_rows=2` 与 `removed=1`。

如果一个类型的 `string`、`vector`、文件流和智能指针成员都能自行管理资源，外层类型通常不应手写析构、复制或移动操作，这就是 Rule of Zero。组合可靠值类型，比维护裸句柄、布尔“是否已关闭”标记和多条清理路径更容易验证。

注Python 迁移提示：Python 的 `with open(path) as f:` 是最接近的资源边界：`__exit__` 在离开块时运行。垃圾回收解决不可达对象的回收，不是及时关闭外部资源的跨实现契约；`__del__` 还会受到循环引用、解释器退出和异常处理限制。C++ 自动对象的析构由词法作用域确定，但动态共享对象仍要等最后一个拥有者消失。

6.3.3 失败诊断与立即练习

析构函数不应让异常逃逸；如果栈展开期间又抛出异常，进程会终止。可能失败的提交或关闭应提供显式操作报告错误，析构只做不抛的尽力清理。另一个常见失败是给已经 Rule of Zero 的类型补一个无必要析构，从而意外影响隐式移动操作。

立即练习：在写入作用域中加入一个条件 `return 0;`，预测文件流是否仍会析构。不要永久保留提前返回；用调试输出或单独实验确认后恢复完整程序。

6.4 值、唯一所有权与共享所有权

6.4.1 选择顺序：先问能否直接拥有值

默认选择顺序是：直接值成员；需要可选堆对象或动态多态时用 `unique_ptr`；只有多个参与者确实共同决定寿命时才用 `shared_ptr`；需要观察共享对象但不延寿时用 `weak_ptr`。智能指针不是“更安全的普通指针”，而是把特定所有权规则编码进类型。

6.4.2 完整输出任务

Listing 6.3: 值、唯一、共享与弱观察状态

```

1  #include <iostream>
2  #include <memory>
3  #include <string>
4  #include <utility>
5
6  struct Ledger final {
7      std::string name;
8  };
9
10 int main() {
11     Ledger direct{"direct"};
12     Ledger copied{direct};
13     copied.name = "copy";
14
15     auto unique = std::make_unique<Ledger>(Ledger{"unique"});

```



```

16     auto transferred = std::move(unique);
17
18     auto shared = std::make_shared<Ledger>(Ledger{"shared"});
19     std::weak_ptr<Ledger> observer{shared};
20
21     std::cout << "direct=" << direct.name << " copied=" << copied.name
22               << " unique_empty=" << (unique == nullptr)
23               << " unique_owner=" << transferred->name
24               << " shared_owners=" << shared.use_count()
25               << " weak_expired=" << observer.expired();
26
27     shared.reset();
28     std::cout << " weak_after_reset=" << observer.expired() << '\n';
29 }

```

Ledger copied{direct} 使用 string 的值语义，修改副本名称不影响原对象。make_unique 创建唯一所有者；std::move(unique) 把所有权交给 transferred，原指针变空。make_shared 建立控制块，weak_ptr 不增加所有者计数，因此 shared_owners=1。调用 shared.reset() 后最后一个强所有者消失，observer.expired() 变为真。

读取弱观察时不能先检查 expired() 再稍后解引用，因为并发环境中对象可能在两步之间销毁；应调用 lock() 一次取得临时 shared_ptr，再判断它是否为空。shared_ptr 的控制块操作通常是线程安全的，但被管理对象的字段并不会自动获得同步。

注Python迁移提示：Python 名称共享对象更像隐式共享引用，但没有在类型中区分唯一、共享和弱观察。weakref 与 weak_ptr 都可观察而不延寿；C++ 则要求接口把所有权意图写得更显式。

6.4.3 失败诊断与立即练习

两个对象互相保存 shared_ptr 会形成强引用环，计数永远不能归零；把不负责延寿的一侧改为 weak_ptr。另一个设计味道是所有参数和成员都改成 shared_ptr：这隐藏了自然所有者、增加原子计数成本，也让对象可能活得远超预期。

立即练习：在输出任务中复制一个 shared_ptr 到内部作用域，预测作用域内外的 use_count()，再运行验证。离开作用域后计数应恢复，不要把具体计数当成并发同步机制。

6.4.4 整合补充：把所有权意图写进函数接口

所有权判断不应只存在于注释里。函数签名是第一道约束：按值参数取得自己的副本或被移动进来的值；const T& 表达调用期间只读借用；T& 表达调用期间可变借用；unique_ptr<T> 按值传入通常表示接管唯一所有权；shared_ptr<T> 按值传入则表示函数要成为共同所有者。若函数只在调用期间读取共享对象，仍应优先接收 const T&，不要为方便而增加引用计数。下面只比较声明，省略类型定义与函数体，不是可单独编译的完整程序：

```

double mark_value(const Quote& quote);           // read-only borrow
void normalize(Quote& quote);                     // mutable borrow
void enqueue(std::unique_ptr<Order> order);       // ownership transfer
void retain(std::shared_ptr<Feed> feed);          // shared lifetime

```

Python 的参数名称和常规类型标注不会在语言层面区分借用、转移与共同延寿，调用约定通常依靠文档和对象行为表达；C++ 接口可以让编译器拒绝复制唯一所有者，却仍不能替程序员证明借用期足够长。

返回值也优先表达值语义。现代 C++ 能通过复制消除或移动高效返回对象，调用者得到清楚的所有权。返回裸指针或引用只有在“被返回对象由别处拥有，且寿命明确长于调用者使用期”时才合理；接口文档还必须说明能否为空。切勿返回局部自动对象的指针或引用，因为函数返回时该对象已经析构。

审查一条接口时可按四问展开：它是否创建对象；谁在调用后拥有对象；借用最长持续到何时；并发任务或回调能否越过该寿命边界。若回答需要“大家小心一点”，类型通常还没有表达真实关系。量化系统里尤其要警惕把栈上行情对象的引用捕获到异步任务：提交函数很快返回，而任务稍后才解引用，形成确定的悬空窗口。

贯穿项目也刻意遵循这条顺序：project/include/quant/types.hpp 让行情、订单、成交和组合快照直接拥有各自的值；回测入口只读借用事件序列与策略，结果则按值返回。项目目前不需要智能指针，这不是遗漏，而是“没有共同动态寿命就不引入共享所有权”的设计证据。

整合练习：只考察“异步保存一份订单”这一处寿命变化。先写按值版本；若订单不可复制，再把同一个参数改为按值接收 `unique_ptr`，并用调用后原指针为空验证所有权已经转移。不要同时引入共享所有权；只有需求明确允许调用者与任务共同延寿时，才另行比较 `shared_ptr` 版本。

6.5 引用、裸指针与底层内存识读

前两节已经建立现代默认路线：先用直接值和 RAII，确有动态寿命再选择智能指针。现在才进入裸指针、C 数组与手动分配；目标是识读遗留接口和诊断内存错误，而不是建立另一套推荐风格。

6.5.1 场景：观察价格不等于拥有价格

引用 `T&` 是已有对象的别名，声明时必须绑定；裸指针 `T*` 保存地址，可以改指向、可以为空。`&price` 取得地址，`*observer` 解引用并访问所指对象，`nullptr` 明确表示没有对象。二者默认都不拥有对象，也不会自动延长被观察对象的生命周期。

6.5.2 完整实验：借用、C 数组与手动释放

Listing 6.4: 非拥有借用与旧式内存语法识读

```

1  #include <iostream>
2
3  void add_half_tick(double& price) { price += 0.5; }
4
5  int main() {
6      double price{100.0};
7      double& alias{price};
8      add_half_tick(alias);
9
10     const double* observer{&price};
11     std::cout << "price=" << price << " alias=" << alias
12               << " observed=" << *observer;
13
14     observer = nullptr;
15     int stack_quotes[3]{100, 101, 102};
16     int* heap_quantity{new int{10}};
17     std::cout << " missing=" << (observer == nullptr)
18               << " stack_last=" << stack_quotes[2]
19               << " heap_quantity=" << *heap_quantity;

```

```

20
21     delete heap_quantity;
22     heap_quantity = nullptr;
23     std::cout << " released=" << (heap_quantity == nullptr) << '\n';
24 }

```

`alias` 与 `price` 是同一对象的两个名字，因此函数通过可变引用把价格从 100 改到 100.5。`observer` 只读观察同一地址；置为 `nullptr` 后，程序先检查再避免解引用。

`int stack_quotes[3]` 是含三个元素的 C 数组，下标有效范围为 0、1、2。它在许多表达式和函数参数中会退化为指向首元素的指针，长度信息不随指针传递；越界不会自动抛出 Python 式异常。现代业务代码默认用 `std::array` 或 `std::vector` 保留大小与值语义，C 数组主要用于识读接口、协议布局和遗留代码。

`new int{10}` 在动态存储期创建整数并返回拥有它的裸指针；`delete heap_quantity` 结束对象生命周期并释放存储。数组形式 `new T[n]` 必须与 `delete[]` 配对，混用释放形式属于未定义行为。这里展示手动配对是为了读懂底层代码，不是鼓励把它作为默认方案。

注Python 迁移提示： Python 名称没有直接的“可空地址并手动解引用”语法。切片和容器通常携带长度；C++ 裸指针不携带长度或所有权。Python 的对象引用也可能让对象继续存活，而 C++ 裸引用和裸指针不会延寿。

6.5.3 失败诊断与立即练习

离开被指对象的作用域后继续解引用会悬空；释放后继续读取是 `use-after-free`；对同一地址再次 `delete` 是 `double-free`。把指针设为空只能防止该变量再次误用，不能修复其他别名。根因分析必须回到唯一的拥有者和销毁时刻。

立即练习：把 `observer = nullptr`；后面增加条件分支，只在非空时读取；分别用空与非空观察者验证。不要直接写 `*observer` 来“看看会发生什么”，未定义行为没有可接受的固定输出。

6.6 失败实验：让 AddressSanitizer 证明悬空

故意失败源码 `code/ch06/failures/use_after_free.cpp` 在 `delete` 之后再次读取同一地址。它与绿色示例隔离，不能进入默认 `all` 目标：

```

1  int* quantity{new int{10}};
2  delete quantity;
3  std::cout << *quantity << '\n'; // intentional use-after-free

```

在 Linux/WSL 中运行：

```

g++ -std=c++20 -O0 -g -fsanitize=address \
    -fno-omit-frame-pointer \
    code/ch06/failures/use_after_free.cpp -o use_after_free
ASAN_OPTIONS=detect_leaks=0:halt_on_error=1 ./use_after_free

```

验收类别是 `heap-use-after-free` 和非零退出，不绑定某一版完整堆栈文本。报告中的“freed by”位置说明存储在哪里释放，“read of size”说明悬空访问发生在哪里。恢复步骤是删除非法读取，并把所有权改为直接值或唯一所有者；只把指针置空不足以修复其他别名。若把非法读取换成第二次 `delete quantity`；，同一所有权错误会表现为 `double-free` 类别。

本项目的 CTest 在 Windows 上通过 WSL GCC 取得真实 ASan 证据；受限环境无法启动 WSL 时测试以明确的 `skip` 状态退出，而发布验收必须在可用的 Linux/WSL 环境实际通过。

6.7 自测、练习与面试表达

1. 对象生命周期的开始和结束分别需要什么事件？为什么存储仍有旧比特不等于对象仍存在？
2. `std::move` 实际做什么？移动后的对象保证什么，又通常不保证什么？
3. 引用、裸指针、`unique_ptr` 和 `shared_ptr` 分别表达何种所有权？
4. C 数组退化后丢失什么信息？`new[]` 为什么必须与 `delete[]` 配对？
5. RAII 如何覆盖异常和提前返回？Rule of Zero 为什么通常比手写析构更稳健？
6. `weak_ptr` 如何打破共享环？为什么“shared”不代表对象字段线程安全？
7. sanitizer 报告 use-after-free 时，如何从分配、释放和非法访问三处恢复生命周期图？

编码练习：

1. 从空文件重建复制/移动轨迹，先写出完整预期输出再运行；答案见 `code/ch06/lifetime_trace.cpp` 与对应 `expected` 文件。
2. 为可空价格观察写出引用版本和指针版本，说明为什么引用版本不能表达“没有价格”；答案见 `code/ch06/borrowing_and_legacy.cpp`。
3. 用文件流作用域写入并读取两行报告，确认资源在第二次打开前已经关闭；答案见 `code/ch06/raii_file.cpp`。
4. 重写本章所有权输出任务并达到章首精确字符串；答案见 `code/ch06/ownership_choices.cpp`。
5. 单独运行 ASan 失败实验，记录非零退出和稳定诊断类别，不把未定义输出当作答案。

问题 6.1 面试检查点 给定“策略异步保存一条行情供稍后读取”，先问谁拥有行情、异步任务最长活多久、能否按值复制，再决定值、唯一所有权或共享所有权。回答必须包含一个悬空失败路径、一个 RAII 清理边界和拒绝滥用 `shared_ptr` 的理由。

6.8 本章小结

生命周期是所有权推理的时间轴：构造建立对象，作用域与拥有者决定析构，引用和裸指针只表达借用。直接值和 Rule of Zero 是默认路径，RAII 把资源释放变成结构性结果；`unique_ptr` 表达可转移的唯一所有权，`shared_ptr` 只用于真实共同寿命，`weak_ptr` 提供不延寿的观察。C 数组与手动分配必须会读、会诊断，但不应取代现代值类型。最后，sanitizer 给出的不是“修复方案”，而是重建错误生命周期图的运行证据。

第七章 错误处理与可观测性

内容提要

- ❑ 区分程序员错误、环境错误和预期业务结果
- ❑ 正确使用异常、显式结果、断言和日志
- ❑ 为 CSV 与 I/O 错误保留上下文
- ❑ 让诊断信息有用而不污染热路径

注 先修知识：理解异常、可选值、文件 I/O 和测试失败。

Python 迁移边界：Python 常用异常覆盖宽泛失败；C++ 热路径更需要区分正常缺席、显式错误与真正异常。

常见错误与诊断：不要吞掉上下文或重复记录；从最外层错误向内检查类别、源行、路径和不变量。

7.1 先分类，再选择机制

程序员错误是被违反的不变量，如负订单数量进入只接受正数量的内部函数；环境错误包括文件不存在、权限不足和数据损坏；“策略没有订单”则是正常业务结果。把三者都表示为异常或布尔值会丢失语义。

本项目用 `optional<OrderIntent>` 表示正常的“无订单”，用显式读取结果表示可能失败的 CSV 边界，用异常终止测试中的失败断言。生产系统可以使用 C++23 `std::expected` 或成熟 backport 表达式/错误联合；C++20 主线在这里保留一个透明的小结果类型。

Listing 7.1: CSV 读取的值或错误边界

```
1  #pragma once
2
3  #include <array>
4  #include <charconv>
5  #include <chrono>
6  #include <cmath>
7  #include <cstdint>
8  #include <istream>
9  #include <string>
10 #include <string_view>
11 #include <vector>
12
13 #include "quant/types.hpp"
14
15 namespace quant {
16
17     struct MarketEventReadResult final {
18         std::vector<MarketEvent> events;
19         std::string error;
20
21         [[nodiscard]] bool has_value() const noexcept { return error.empty(); }
22     };
23
24     namespace detail {
25
26         inline bool split_row(std::string_view row,
27                               std::array<std::string_view, 4>& fields) {
```

```

28     for (std::size_t index = 0; index < fields.size(); ++index) {
29         const auto comma = row.find(',');
30         if (index + 1 == fields.size()) {
31             if (comma != std::string_view::npos) {
32                 return false;
33             }
34             fields[index] = row;
35             return true;
36         }
37         if (comma == std::string_view::npos) {
38             return false;
39         }
40         fields[index] = row.substr(0, comma);
41         row.remove_prefix(comma + 1);
42     }
43     return true;
44 }
45
46 template <typename Number>
47 inline bool parse_number(std::string_view text, Number& value) {
48     const char* begin = text.data();
49     const char* end = begin + text.size();
50     const auto [position, error] = std::from_chars(begin, end, value);
51     return error == std::errc{} && position == end;
52 }
53
54 } // namespace detail
55
56 inline MarketEventReadResult read_market_events(std::istream& input) {
57     MarketEventReadResult result;
58     std::string row;
59     if (!std::getline(input, row)) {
60         result.error = "line 1: missing CSV header";
61         return result;
62     }
63     if (row != "timestamp_ms,symbol,price,quantity") {
64         result.error = "line 1: expected timestamp_ms,symbol,price,quantity";
65         return result;
66     }
67
68     std::size_t line = 1;
69     while (std::getline(input, row)) {
70         ++line;
71         if (row.empty()) {
72             continue;
73         }
74         std::array<std::string_view, 4> fields{};
75         std::int64_t timestamp_ms = 0;
76         std::int64_t quantity = 0;

```



```

77     double price = 0.0;
78     if (!detail::split_row(row, fields) || fields[1].empty() ||
79         !detail::parse_number(fields[0], timestamp_ms) ||
80         !detail::parse_number(fields[2], price) ||
81         !detail::parse_number(fields[3], quantity) || !std::isfinite(price) ||
82         price <= 0.0 || quantity <= 0) {
83         result.events.clear();
84         result.error = "line " + std::to_string(line) + ": invalid market event";
85         return result;
86     }
87     result.events.push_back(MarketEvent{
88         Timestamp{std::chrono::milliseconds{timestamp_ms}},
89         std::string{fields[1]}, price, quantity});
90 }
91 return result;
92 }
93
94 } // namespace quant

```

7.2 错误必须携带上下文

“invalid argument”几乎无法操作。读取器报告源行号，并在头部不匹配时给出期望格式。更完整的系统还会加入文件路径、字段名和原始值摘要，但应避免把敏感数据或整行巨大输入直接写入日志。

错误沿调用栈传播时，每个知道新信息的边界补充一层上下文，而不是重复记录同一错误。通常由最外层拥有请求或任务标识的层写一次日志，底层返回结构化错误。

7.3 异常的正确位置

异常适合无法在当前抽象层处理、且失败路径相对少见的情况。**RAII** 让栈展开安全释放资源。热循环不应使用异常表达每个 tick 上的普通分支，因为抛出路径昂贵、控制流不明显，也会干扰延迟分布。

公开库是否抛异常还受到 **ABI**、编译选项和团队规范影响。关键是一个边界只采用一套清楚策略，并记录哪些操作可能失败、失败后对象保持什么状态。

7.4 断言与验证

断言用于“正确程序不可能违反”的内部条件，不能替代外部输入验证，因为 **Release** 可能移除断言。测试同样不能依赖会被 **NDEBUG** 移除的 **assert**；本项目使用始终执行的检查函数，正是一次真实构建发现的教训。

7.5 日志、指标与追踪

日志回答发生了什么，指标回答发生频率与趋势，追踪连接一次请求或事件的跨组件路径。低延迟热路径可将结构化事件写入预分配缓冲区，由后台线程批量输出；任何方案都应定义缓冲满时丢弃、阻塞还是降级。

不要在基准循环中同步格式化大量日志后声称测得算法延迟。基准应关闭非必要观测，生产则保留足以定位失败的低开销信号。

问题 7.1 面试检查点 “无订单”为什么不应是异常？断言与输入验证有何区别？错误在哪一层记录日志最好？热路径日志缓冲满时有哪些策略与风险？

7.6 练习

1. 为 CSV 头部顺序错误、NaN 价格和零数量各写一个行为测试。
2. 设计一个结构化错误类型，包含类别、行号、字段和消息，比较它与字符串错误的演化成本。
3. 为撮合拒绝设计原因枚举，讨论返回空值何时信息不足。
4. 写出回测任务的三项指标和两条仅在失败时记录的日志。

7.7 本章小结

错误机制应匹配错误类别。正常缺席用可选值，外部失败用显式结果或异常，内部不变量用断言；上下文在边界补充，日志只记录一次并兼顾热路径成本。

第八章 CMake、测试与代码质量

内容提要

- ❑ 用 target-based CMake 表达构建关系
- ❑ 组织警告、sanitizer、静态分析和 CI 配置
- ❑ 在公开 seam 编写不依赖内部实现的行为测试
- ❑ 让研究代码具备可重复构建路径

注 先修知识：能使用 Python 测试框架和虚拟环境，理解公开接口。

Python 迁移边界： CMake target 类似可复现环境声明，但还决定编译、链接与传递依赖；CTest 必须在 Release 下仍执行真实断言。

常见错误与诊断：绿色但无断言是测试基础设施故障；检查测试发现数、失败敏感性、编译配置和干净构建。

8.1 构建系统描述目标

现代 CMake 的中心是目标。每个可执行文件或库声明自己的源文件、头文件搜索路径、编译特性和依赖；PRIVATE、PUBLIC 与 INTERFACE 控制需求是否传播。避免全局加入所有 include 路径和编译选项，这会让无关目标互相污染。

```
cmake -S . -B build-mingw -G "MinGW Makefiles" \  
      -DCMAKE_BUILD_TYPE=Release  
cmake --build build-mingw  
ctest --test-dir build-mingw --output-on-failure
```

Linux 上推荐 Ninja 或 Unix Makefiles，编译器由工具链文件或环境选择。团队应保存可重复命令、依赖版本和至少一种受支持配置，不要求每位开发者手工点击 IDE 才能构建。

8.2 测试公开行为

本项目预先确认四个 seam：行情输入、策略决策、成交与组合、整段回测。测试只调用公开接口，不访问私有持仓表，也不验证某函数内部调用次数。这样数据结构和算法可以重构，而业务行为仍受保护。

TDD 循环是一条竖直切片：写一个失败行为，确认失败原因正确，只实现足够代码让它通过，再进入下一行为。一次写几十个基于想象的测试，常会锁定错误接口。

8.3 独立预期值

测试预期应来自手算例子、规范或可信外部数据，而不是把实现公式复制进测试。回测例子用三笔已知价格手算：以 99 买入 25 股，从 10000 现金出发，末价 103 时权益为

$$10000 - 25 \times 99 + 25 \times 103 = 10100,$$

总收益为 1%。这个字面预期可以与实现真正分歧。

8.4 测试也会失效

Release 配置通常定义 NDEBUG，会移除标准 assert。本项目最初的 seam 测试曾因此“绿色但未断言”，编译器的 unused 警告揭示了问题。测试框架或自定义检查必须在所有测试配置中保持有效。

8.5 质量工具分工

编译器警告发现可疑转换和未使用代码；sanitizer 观察执行路径上的内存与 UB；静态分析覆盖未运行路径和代码模式；格式化器减少无意义差异；代码审查检查契约、命名和范围。它们互补，任何一个绿色都不能证明程序无错。

建议 CI 矩阵至少包含：GCC/Clang Debug + 测试，sanitizer 配置，Release + 基准烟雾测试。性能回归阈值要考虑共享机器噪声，严肃延迟验证应在固定硬件和受控负载上进行。

8.6 端到端样例

Listing 8.1: 从 CSV 到回测摘要的命令行程序

```

1  #include <fstream>
2  #include <iomanip>
3  #include <iostream>
4  #include <string>
5
6  #include "quant/backtest.hpp"
7  #include "quant/csv.hpp"
8
9  int main(int argc, char** argv) {
10     if (argc != 2) {
11         std::cerr << "usage: quant_backtest <market-data.csv>\n";
12         return 2;
13     }
14
15     std::ifstream input{argv[1]};
16     if (!input) {
17         std::cerr << "cannot open market data: " << argv[1] << '\n';
18         return 2;
19     }
20     const auto parsed = quant::read_market_events(input);
21     if (!parsed.has_value()) {
22         std::cerr << parsed.error << '\n';
23         return 2;
24     }
25
26     const quant::ThresholdStrategy strategy{100.0, 25};
27     const quant::BacktestEngine engine{10'000.0};
28     const auto result = engine.run(parsed.events, strategy);
29     std::cout << "fills=" << result.fills.size() << " equity="
30         << result.final_portfolio.equity << " return=" << std::fixed
31         << std::setprecision(0) << result.performance.total_return * 100.0
32         << "% volatility=" << std::setprecision(6)
33         << result.performance.volatility << " trades="
34         << result.performance.trades.fill_count << '\n';
35 }

```

这个程序在进程边界处理文件打开和解析错误，核心引擎只接收已验证事件。它很小，却贯通输入、策略、撮合、组合与报告，是比大量孤立类更有价值的 `tracer bullet`。

问题 8.1 面试检查点 为什么 `target-based CMake` 优于全局 `flags`？什么是测试 `seam`？为什么复制实现公式会产生恒真测试？`sanitizer` 绿色可以证明什么、不能证明什么？

8.7 练习

1. 新增 `Debug + ASan/UBSan` 配置并运行全部测试。
2. 故意改变撮合价格，确认端到端测试以有意义消息失败。
3. 将一个测试改写为只验证公开结果，不检查内部容器。
4. 设计一份 `CI` 矩阵，说明每个 `job` 捕获的失败类别和成本。

8.8 本章小结

`CMake` 让构建关系显式，测试在公开 `seam` 保护行为，多类工具提供互补证据。可靠工程的关键不是工具数量，而是每个检查都真实执行、失败可诊断、从干净环境可重复。

第三部分

量化性能工程

第九章 数据布局、缓存与分配

内容提要

- ❑ 用缓存行和局部性解释真实访问成本
- ❑ 识别分配、对象大小和间接访问
- ❑ 比较 AoS 与 SoA，而不预设结论
- ❑ 选择测量后才值得引入的内存优化

注 先修知识：理解数组、对象布局、缓存层次与前章构建流程。

Python 迁移边界：NumPy 的连续数组已体现局部性；C++ 还允许直接选择字段布局、对齐和分配策略。

常见错误与诊断：不要从复杂度或一次计时推断缓存行为；检查对象大小、访问字段、硬件计数器与重复样本。

9.1 复杂度相同，耗时仍可不同

两个 $O(n)$ 循环可能相差很大。CPU 以缓存行为单位搬运数据，连续访问允许硬件预取；指针追逐、过大的对象和随机访问会增加缓存未命中。大 O 描述规模增长，不描述常数、布局或尾延迟。

在热循环中，先列出真正读取的字段。如果计算只需要价格和数量，但每个元素还携带字符串、时间戳和元数据，Array of Structures (AoS) 会把无关字节一起搬入缓存。Structure of Arrays (SoA) 把字段分列，适合扫描少数字段；但处理完整单条事件时，AoS 可能更自然且局部。

Listing 9.1: 对同一工作负载比较 AoS 与 SoA

```
1 #include <algorithm>
2 #include <chrono>
3 #include <cstdint>
4 #include <iostream>
5 #include <vector>
6
7 struct Tick final {
8     double price;
9     std::int64_t quantity;
10    std::int64_t timestamp;
11 };
12
13 template <typename Work>
14 auto measure(Work&& work) {
15     const auto start = std::chrono::steady_clock::now();
16     const double checksum = work();
17     const auto elapsed = std::chrono::steady_clock::now() - start;
18     return std::pair{checksum,
19                     std::chrono::duration_cast<std::chrono::microseconds>(
20                         elapsed)
21                     .count()};
22 }
23
24 double median(std::vector<long long> samples) {
25     std::ranges::sort(samples);
26     const std::size_t middle = samples.size() / 2;
```

```

27     return samples.size() % 2 == 0
28         ? (static_cast<double>(samples[middle - 1]) +
29            static_cast<double>(samples[middle])) /
30            2.0
31         : static_cast<double>(samples[middle]);
32 }
33
34 long long interquartile_range(std::vector<long long> samples) {
35     std::ranges::sort(samples);
36     return samples[(samples.size() * 3) / 4] - samples[samples.size() / 4];
37 }
38
39 int main() {
40     constexpr std::size_t size = 1'000'000;
41     std::vector<Tick> aos;
42     std::vector<double> prices;
43     std::vector<std::int64_t> quantities;
44     aos.reserve(size);
45     prices.reserve(size);
46     quantities.reserve(size);
47     for (std::size_t i = 0; i < size; ++i) {
48         const double price = 90.0 + static_cast<double>(i % 200) * 0.01;
49         const std::int64_t quantity = 1 + static_cast<std::int64_t>(i % 100);
50         aos.push_back(Tick{price, quantity, static_cast<std::int64_t>(i)});
51         prices.push_back(price);
52         quantities.push_back(quantity);
53     }
54
55     const auto run_aos = [&] {
56         double sum = 0.0;
57         for (const Tick& tick : aos) {
58             sum += tick.price * static_cast<double>(tick.quantity);
59         }
60         return sum;
61     };
62     const auto run_soa = [&] {
63         double sum = 0.0;
64         for (std::size_t i = 0; i < prices.size(); ++i) {
65             sum += prices[i] * static_cast<double>(quantities[i]);
66         }
67         return sum;
68     };
69
70     // Warm both paths before collecting samples.
71     const double warm_aos = run_aos();
72     const double warm_soa = run_soa();
73     constexpr int iterations = 20;
74     std::vector<long long> aos_samples;
75     std::vector<long long> soa_samples;

```



```

76  aos_samples.reserve(iterations);
77  soa_samples.reserve(iterations);
78  double checksum_guard = warm_aos + warm_soa;
79  bool checksums_match = warm_aos == warm_soa;
80
81  for (int iteration = 0; iteration < iterations; ++iteration) {
82      // Alternate order to reduce a fixed first/second measurement bias.
83      if (iteration % 2 == 0) {
84          const auto [aos_sum, aos_us] = measure(run_aos);
85          const auto [soa_sum, soa_us] = measure(run_soa);
86          aos_samples.push_back(aos_us);
87          soa_samples.push_back(soa_us);
88          checksums_match = checksums_match && aos_sum == soa_sum;
89          checksum_guard += aos_sum + soa_sum;
90      } else {
91          const auto [soa_sum, soa_us] = measure(run_soa);
92          const auto [aos_sum, aos_us] = measure(run_aos);
93          aos_samples.push_back(aos_us);
94          soa_samples.push_back(soa_us);
95          checksums_match = checksums_match && aos_sum == soa_sum;
96          checksum_guard += aos_sum + soa_sum;
97      }
98  }
99
100  std::cout << "iterations=" << iterations
101             << " aos_median_us=" << median(aos_samples)
102             << " aos_iqr_us=" << interquartile_range(aos_samples)
103             << " soa_median_us=" << median(soa_samples)
104             << " soa_iqr_us=" << interquartile_range(soa_samples)
105             << " checksum-match=" << std::boolalpha << checksums_match
106             << " checksum=" << checksum_guard << '\n';
107 }

```

这个程序先预热两条路径，再以交替顺序各测量 20 次，报告中位数与四分位距，并验证每轮 checksum 一致。它仍是固定数据规模、未固定 CPU 频率的本机微基准，不能据此发布跨机器结论；原始环境与完整命令必须随结果保存。

9.2 对象大小与对齐

成员顺序会引入填充。使用 `sizeof` 和 `alignof` 观察类型，而不要凭字段和相加猜测。把常用小字段集中可能缩小对象，但为了省几个字节破坏清晰语义并不总值得。网络或磁盘协议布局应显式序列化，不能把内存结构直接当稳定格式。

9.3 分配是系统行为

动态分配包含查找可用块、同步、元数据和潜在缺页。`vector::reserve` 可减少可预测增长中的重复分配。对象池和 `std::pmr` 资源适合分配模式已知且测得分配热点的场景，但会引入生命周期、峰值容量和线程归属问题。

短生命周期批量对象可以使用 `arena`，在整批结束时统一释放。前提是任何引用都不能逃出 `arena` 生命周期；否则“更快释放”换来了悬空风险。

9.4 false sharing

两个线程写不同变量，如果变量落在同一缓存行，缓存一致性仍会在核心间来回转移该行。按线程分片、减少共享写入或适当填充可以缓解。盲目添加 `alignas(64)` 会增大内存并假设硬件缓存行，应在目标平台测量。

问题 9.1 面试检查点 AoS 与 SoA 各适合什么访问模式？为什么链表的常数时间插入不保证更快？对象池增加了哪些正确性风险？什么是 false sharing？

9.5 练习

1. 在三个独立进程中运行布局基准，比较各自中位数与 IQR，解释进程间漂移。
2. 删除 AoS 的时间戳字段后重测，解释对象大小与结果变化。
3. 用 `profiler` 验证分配是否真是 CSV 读取热点，再决定是否引入预留容量。
4. 设计按线程分片的计数器并说明最终归约时机。

9.6 本章小结

性能来自访问模式与硬件层次的互动。连续布局、减少无关字节和控制分配是有力工具，但都要以工作负载和测量为依据。

第十章 Benchmark、profiling 与优化方法

内容提要

- ❑ 把性能问题写成可反驳假设
- ❑ 正确处理预热、噪声、优化器和统计摘要
- ❑ 区分微基准、端到端基准与 profiler
- ❑ 用性能预算而不是“越快越好”指导取舍

注 先修知识：能运行 Release 构建并理解基本统计摘要。

Python 迁移边界：Python 的 `timeit` 与 `profiler` 体现同一实验原则：跨语言结果都必须定义工作负载和消费输出。

常见错误与诊断：性能结论冲突时检查预热、顺序、死代码删除、CPU 状态、样本分布和端到端占比。

10.1 优化循环

可重复的优化循环是：定义用户可见指标，记录基线，定位热点，提出单一假设，做最小改动，重新测量，并验证正确性没有回归。若结果落在噪声范围内，结论是“证据不足”，不是挑最好的一次宣布成功。

吞吐量适合批量历史数据；平均延迟会掩盖尾部；交易系统常关注 p_{50} 、 p_{99} 、 $p_{99.9}$ 和最大值，同时记录负载和丢弃策略。任何百分位都必须说明样本量、采样边界和计时器。

10.2 微基准的陷阱

优化器可能删除未使用结果，因此基准应消费 `checksum`；首次运行包含页面映射和缓存冷启动；CPU 动态频率、后台任务和安全软件带来噪声；计时器调用本身也有成本。微基准适合隔离机制，不代表整套系统收益。

本书的布局程序刻意同时输出结果一致性、20 次交替顺序测量的中位数和 `IQR`，并在采样前预热。更严谨的实验还应保存原始样本、固定或记录 CPU 状态、跨进程重复并在固定硬件上运行。Google Benchmark 等框架能处理部分细节，但不能替你定义有意义工作负载。

10.3 profiler 回答“时间在哪里”

采样 `profiler` 以较低扰动发现 CPU 热点和调用栈；`instrumentation` 可提供精确事件但扰动更大；硬件计数器观察周期、指令、分支、缓存未命中。先用低成本方法定位，再针对假设选择证据。

如果函数占总时间 5%，即使把它变为零，整体最多约提升 $1/(1 - 0.05) \approx 1.053$ 倍。Amdahl 定律提醒我们不要优化醒目的小函数而忽略真正瓶颈。

10.4 端到端预算

回测性能可以分解为读取、解析、事件分发、策略、撮合、簿记和统计。实时路径还包括网络、排队、序列化与系统调用。为每段定义预算和观测点，才能判断局部微基准是否移动整体指标。

10.5 性能与可维护性

若两种实现差异小于噪声，选择更清楚者。若优化显著但复杂，应记录适用工作负载、基准命令和回退方案，并用回归基准保护。不要把硬件相关技巧扩散到业务层；将其封装在有普通参考实现的深模块里。

问题 10.1 面试检查点 为什么 `checksum` 能阻止部分死代码删除？微基准与端到端基准回答什么不同问题？如何解释 p_{99} ？Amdahl 定律如何改变优化优先级？

10.6 练习

1. 为布局基准保存 20 个原始样本，并增加最小值、最大值和跨进程摘要。
2. 用 `profiler` 检查回测样例，提出一个可被数据推翻的瓶颈假设。
3. 定义每秒事件数与 p_{99} 延迟的双重性能预算。
4. 分析一个优化如果使代码复杂一倍但只提升 2%，需要哪些业务证据才值得保留。

10.7 本章小结

性能工程是一种实验方法。指标、基线、热点、假设、改动和复测必须形成闭环；微基准提供局部证据，`profiler` 定位系统成本，端到端预算决定优化是否有价值。

第十一章 并发、原子与任务系统

内容提要

- ❑ 从所有权和 happens-before 推理并发正确性
- ❑ 设计启动、停止、背压和异常传播协议
- ❑ 区分线程、任务、互斥锁与原子变量
- ❑ 避免把“无锁”误解为“无等待且更快”

注 先修知识：理解线程、作用域、RAII 与不可变输入。

Python 迁移边界：Python 线程常受 GIL 约束；C++ 线程可真正并行，也会因未同步冲突直接触发数据竞争 UB。

常见错误与诊断：偶发错误先用所有权图、happens-before 与 ThreadSanitizer 定位，不用 sleep 掩盖竞态。

11.1 先问为什么并发

并发可能为了并行计算、隐藏 I/O 等待或隔离独立职责。加入线程前先确认工作可分解、数据依赖允许，以及收益大于同步和调度成本。小型回测常被内存带宽限制，增加核心未必线性加速。

数据竞争发生在多个线程并发访问同一内存位置、至少一个写入且缺少同步时，它属于 UB。即使某次运行结果正确，也没有语言保证。

11.2 不共享可变状态

最容易证明的设计是每个任务拥有自己的输出，线程结束后由单线程归约：

Listing 11.1: 分片归约与结构化 join

```
1  #include <cstdint>
2  #include <iostream>
3  #include <numeric>
4  #include <thread>
5  #include <vector>
6
7  int main() {
8      std::vector<std::int64_t> values(100'000);
9      std::iota(values.begin(), values.end(), std::int64_t{1});
10
11     std::int64_t left_sum = 0;
12     std::int64_t right_sum = 0;
13     const auto middle = values.begin() + static_cast<std::ptrdiff_t>(values.size() / 2);
14     {
15         std::jthread left([&] {
16             left_sum = std::accumulate(values.begin(), middle, std::int64_t{0});
17         });
18         std::jthread right([&] {
19             right_sum = std::accumulate(middle, values.end(), std::int64_t{0});
20         });
21     } // jthread destructors join before the two results are read
22 }
```

```

23     std::cout << "sum=" << left_sum + right_sum << '\n';
24 }

```

两个线程只读共同输入，但分别写 `left_sum` 和 `right_sum`。jthread 析构时 join，主线程在读取结果前获得同步边界。若线程写相邻字段，仍应测量 false sharing。

11.3 互斥锁与原子

互斥锁保护一组不变量，适合现金与持仓必须一致更新的复合状态。原子变量适合单个状态或可严格定义的无锁协议；把多个字段分别改成 `atomic` 不会自动形成一致快照。

内存序决定哪些读写对其他线程可见。除非能画出同步关系并证明更弱序安全，使用默认顺序或锁通常更可靠。低延迟代码中的错误内存序可能只在特定 CPU 和负载出现。

11.4 队列、背压与停止

生产者快于消费者时，队列必然增长、阻塞或丢弃。系统必须显式选择容量和满载策略，并让指标观察积压。无界队列把背压问题推迟成内存耗尽。

停止协议应回答：谁发出停止、生产者何时停止接收、消费者是否排空、阻塞等待如何唤醒、异常如何传回拥有线程。jthread 与 `stop_token` 提供协作取消机制，但被调用代码仍需定期检查并安全退出。

11.5 任务系统

线程是执行资源，任务是工作单元。线程池减少反复创建线程，工作窃取改善不均匀任务负载。任务粒度太小会让调度占主导，太大则降低并行度。量化计算可按日期、资产或参数分片，但必须处理确定性归约和共享缓存。

11.6 无锁不是默认目标

lock-free 意味着系统整体持续前进，不保证每个线程有界等待；wait-free 才提供每个操作的步数界。无锁结构还涉及 ABA、内存回收和伪共享。先用锁建立正确基线，用 profiler 证明争用，再考虑成熟实现。

问题 11.1 面试检查点 数据竞争为何是 UB？多个 `atomic` 字段能否构成一致事务？有界队列满时有哪些策略？lock-free 与 wait-free 有何区别？

11.7 练习

1. 扩展并行归约为按资产分片，确保每个线程独占输出。
2. 为有界行情队列写出阻塞、丢弃最新和覆盖最旧三种策略的业务后果。
3. 画出生产者、消费者和控制线程的停止时序图。
4. 用 ThreadSanitizer（支持的平台上）验证一个故意引入的共享写错误，再修复。

11.8 本章小结

并发正确性来自清晰所有权和同步边，而不是原子数量。优先分片、消息传递和结构化生命周期；队列必须有背压，线程必须有停止协议，无锁优化必须由争用证据驱动。

第十二章 数值计算与 Python 互操作

内容提要

- ❑ 理解浮点表示、舍入与误差传播
- ❑ 设计清楚的 Python/C++ 所有权和数组边界
- ❑ 使用稳定归约与容差测试
- ❑ 判断何时值得把热点移入 C++

注 先修知识：理解 Python/NumPy 数组、浮点基础和 C++ 生命周期。

Python 迁移边界：NumPy 已调用原生内核；只有 Python 调度或逐元素循环真是热点时，跨语言批量边界才可能获益。

常见错误与诊断：数值差异先检查表示、归约顺序和容差；绑定崩溃则检查 dtype、连续性、GIL 与缓冲所有权。

12.1 浮点不是实数

IEEE 754 二进制浮点只能精确表示有限集合。多数十进制小数会被舍入；加法不满足结合律；数量级相差悬殊时，小值可能消失。直接用 `==` 比较经计算结果通常不合适，但容差也必须结合量纲和误差模型。

经典灾难例子是 $10^{16} + 1 - 10^{16}$ ：按双精度顺序累加可能得到 0。补偿求和保留舍入中丢失的一部分：

Listing 12.1: 普通求和与补偿求和

```
1 #include <cmath>
2 #include <iostream>
3 #include <vector>
4
5 double compensated_sum(const std::vector<double>& values) {
6     double sum = 0.0;
7     double correction = 0.0;
8     for (const double value : values) {
9         const double next = sum + value;
10        if (std::abs(sum) >= std::abs(value)) {
11            correction += (sum - next) + value;
12        } else {
13            correction += (value - next) + sum;
14        }
15        sum = next;
16    }
17    return sum + correction;
18 }
19
20 int main() {
21     const std::vector<double> values{1e16, 1.0, -1e16};
22     double naive = 0.0;
23     for (const double value : values) {
24         naive += value;
25     }
26     std::cout << "naive=" << naive
27               << " compensated=" << compensated_sum(values) << '\n';
```



```
28 }
```

补偿算法提高精度但增加运算和依赖链，并非所有工作负载都更快。先定义允许误差，再在真实数据分布上测试。

12.2 价格、收益与方差

交易金额常需要可审计舍入，可用整数最小货币单位或定点表示；收益率和模型计算通常需要浮点。跨表示转换必须集中并写明舍入模式。在线方差应使用数值稳定算法，而不是分别累计 $\sum x$ 与 $\sum x^2$ 后相减。

测试浮点结果时可使用绝对与相对容差组合：

$$|a - b| \leq \epsilon_{abs} + \epsilon_{rel} \max(|a|, |b|).$$

容差不是让测试变绿的旋钮，应来自业务最小显著差异和算法误差分析。

12.3 先确认 Python 热点在哪里

NumPy、SciPy 和许多 pandas 操作已经在 C/C++/Fortran 中执行。把外层 Python 函数重写为 C++ 可能没有收益，反而增加复制与绑定成本。profiler 应区分 Python 调度、原生内核、内存分配和数据转换。

12.4 pybind11 边界

pybind11 适合暴露小而稳定的 C++ API。边界必须说明：数组是否连续、dtype 与字节序、谁拥有内存、调用期间能否释放 GIL、异常如何映射、返回视图能活多久。

Listing 12.2: 可编译的 pybind11 可选边界

```
1  #include <iostream>
2  #include <stdexcept>
3  #include <vector>
4
5  [[nodiscard]] double final_marked_equity(const std::vector<double>& prices,
6                                          double initial_cash) {
7      if (prices.empty()) {
8          throw std::invalid_argument("prices must not be empty");
9      }
10     return initial_cash + (prices.back() - prices.front());
11 }
12
13 #ifdef QUANT_WITH_PYBIND11
14 #include <pybind11/pybind11.h>
15 #include <pybind11/stl.h>
16
17 PYBIND11_MODULE(quant_boundary, module) {
18     module.def("final_marked_equity", &final_marked_equity,
19              pybind11::arg("prices"), pybind11::arg("initial_cash"));
20 }
21 #else
22 int main() {
```

```

23     const std::vector<double> prices{99.0, 101.0, 103.0};
24     std::cout << "pybind-boundary final-equity="
25               << final_marked_equity(prices, 99.0) << '\n';
26 }
27 #endif

```

该源文件始终作为 C++20 可执行目标编译并验证边界函数；若 CMake 找到 pybind11，还会额外构建名为 `quant_boundary` 的真实 Python 模块。这样默认教程不强制下载依赖，安装依赖的环境又会编译绑定代码，而不是让书中草图长期漂移。

对 NumPy 数组，零拷贝视图要求底层缓冲在 C++ 使用期间存活；返回指向临时 `vector` 的数组是悬空错误。若 C++ 长计算不访问 Python 对象，可释放 GIL，但重新调用 Python 前必须获取。

12.5 批量边界胜过逐元素调用

跨语言调用具有固定成本。每个 tick 从 Python 调一次 C++，往往不如一次传入连续批次；反过来，若实时回调由 C++ 驱动，则策略接口和 GIL 行为要单独设计。边界应传递领域批次，而不是暴露大量细碎 `getter`。

问题 12.1 面试检查点 为什么浮点加法不满足结合律？金额与收益率为何可能选择不同表示？零拷贝 NumPy 视图需要什么生命周期保证？何时可以释放 GIL？

12.6 练习

1. 为收益率比较定义绝对/相对容差，并解释数值来源。
2. 用 Welford 算法实现在线方差，与两遍算法比较。
3. 设计批量回测绑定，列出 `shape`、`dtype`、所有权和错误契约。
4. profiler 一个 NumPy 工作流，判断瓶颈是否真的位于 Python 循环。

12.7 本章小结

数值正确性需要表示、算法和容差共同定义。Python/C++ 互操作的核心是批量边界和生命周期，不是绑定语法；只有测得热点且接口稳定时，迁移到 C++ 才有工程价值。

第四部分

求职到工作

第十三章 贯穿项目：事件驱动回测引擎

内容提要

- ❑ 把输入、策略、撮合、组合与统计组装为完整数据流
- ❑ 识别教学引擎与生产平台之间的明确边界
- ❑ 把项目讲成可复现的求职作品，而不是代码堆积
- ❑ 用公开 seam 验证组件与端到端行为

注 先修知识：完成前 12 章并能运行 CMake/CTest。

Python 迁移边界：Python 原型常在一处隐式共享状态；贯穿项目把输入、决策、事实与统计拆成显式值和 seam。

常见错误与诊断：端到端结果错误时沿数据流逐段检查，先用手算账本确定第一个发生偏差的公开边界。

13.1 从一个事件开始

贯穿项目的数据流是

CSV → MarketEvent → Strategy → OrderIntent → Fill → Portfolio → BacktestResult.

每条箭头都是边界：CSV 解析将不可信文本变为已验证值；策略只观察事件和只读组合快照；撮合器决定订单能否成为成交；组合只接受事实成交；独立统计模块汇总总收益、最大回撤、逐事件收益波动与交易摘要。

这种设计避免“策略直接改现金”或“CSV 行同时充当领域对象”。边界越清楚，越容易替换数据源、策略或撮合模型，也更容易在正确位置测试错误。

13.2 运行项目

```
cmake -S . -B build-mingw -G "MinGW Makefiles" \  
      -DCMAKE_BUILD_TYPE=Release  
cmake --build build-mingw  
ctest --test-dir build-mingw --output-on-failure  
./build-mingw/quant_backtest.exe project/data/sample.csv
```

样例从 10000 现金出发，在价格 99 时买入 25 股，末价为 103，输出一笔成交、权益 10100、1% 总收益和非年化逐事件波动。这个结果来自独立手算，是端到端测试的事实来源。

13.3 绩效统计契约

Listing 13.1: 收益、回撤、波动与交易摘要

```
1 #pragma once  
2  
3 #include <algorithm>  
4 #include <cmath>  
5 #include <cstdint>  
6 #include <cstdint>  
7 #include <vector>  
8
```

```

9  #include "quant/types.hpp"
10
11 namespace quant {
12
13 struct TradeSummary final {
14     std::size_t fill_count{};
15     std::int64_t buy_quantity{};
16     std::int64_t sell_quantity{};
17     double gross_notional{};
18 };
19
20 struct PerformanceSummary final {
21     double total_return{};
22     double max_drawdown{};
23     double volatility{}; // sample standard deviation of per-event returns
24     TradeSummary trades;
25 };
26
27 inline PerformanceSummary summarize_performance(
28     double initial_cash, const std::vector<double>& equity_curve,
29     const std::vector<Fill>& fills) {
30     PerformanceSummary summary;
31     summary.trades.fill_count = fills.size();
32     for (const Fill& fill : fills) {
33         if (fill.side == Side::buy) {
34             summary.trades.buy_quantity += fill.quantity;
35         } else {
36             summary.trades.sell_quantity += fill.quantity;
37         }
38         summary.trades.gross_notional +=
39             fill.price * static_cast<double>(fill.quantity);
40     }
41
42     if (equity_curve.empty()) {
43         return summary;
44     }
45     if (initial_cash != 0.0) {
46         summary.total_return = equity_curve.back() / initial_cash - 1.0;
47     }
48
49     double peak = initial_cash;
50     std::vector<double> returns;
51     returns.reserve(equity_curve.size() > 1 ? equity_curve.size() - 1 : 0);
52     for (std::size_t index = 0; index < equity_curve.size(); ++index) {
53         const double equity = equity_curve[index];
54         peak = std::max(peak, equity);
55         if (peak > 0.0) {
56             summary.max_drawdown =
57                 std::max(summary.max_drawdown, (peak - equity) / peak);

```

```

58     }
59     if (index > 0 && equity_curve[index - 1] != 0.0) {
60         returns.push_back(equity / equity_curve[index - 1] - 1.0);
61     }
62 }
63
64 if (returns.size() >= 2) {
65     double mean = 0.0;
66     for (const double value : returns) {
67         mean += value;
68     }
69     mean /= static_cast<double>(returns.size());
70     double squared_deviation = 0.0;
71     for (const double value : returns) {
72         const double deviation = value - mean;
73         squared_deviation += deviation * deviation;
74     }
75     summary.volatility =
76         std::sqrt(squared_deviation / static_cast<double>(returns.size() - 1));
77 }
78 return summary;
79 }
80
81 } // namespace quant

```

本项目的 `volatility` 是相邻权益点简单收益的样本标准差，不做年化，因为输入事件没有固定频率。交易摘要包含成交数、买卖数量和成交总名义金额。生产系统必须进一步定义时间加权、现金流、基准、年化频率、无风险利率和费用口径。

13.4 四个测试 seam

1. 行情输入：文本要么成为规范事件，要么给出带行号错误。
2. 策略决策：给定事件和快照，产生订单或正常的空值。
3. 成交与组合：订单按明确模型成交，快照显示一致现金和持仓。
4. 整段回测：已知事件序列产生确定成交与绩效摘要。

这些测试通过公开接口观察行为。持仓内部使用哈希表只是实现细节；更换容器不应让测试失败。相反，若成交价格或现金符号改变，行为测试必须失败。

13.5 当前引擎明确不做什么

教学引擎只处理单资产快照、市场事件驱动的立即成交、无手续费、无滑点、无延迟和单币种现金。它没有公司行动、交易日历、订单生命周期、部分成交、借券、保证金、复权、数据修订或生存者偏差控制。

这些限制不是脚注。若把样例收益解释为可交易结果，就犯了模型风险错误。生产演进应先选择一个真实用例，再增加对应契约和测试。例如支持手续费时，成交模型产生费用明细，组合一次性提交现金、持仓与费用，端到端预期由独立账本例子给出。

13.6 扩展顺序

合理演进路线如下：

1. 多资产估值：引入价格映射和缺失价格政策。
2. 订单生命周期：订单 ID、状态、部分成交、撤单和拒绝原因。
3. 成本模型：手续费、点差、滑点与容量限制。
4. 数据正确性：交易日历、时区、复权、公司行动和版本化数据。
5. 性能：批量解析、列式事件、分片回测和受控基准。
6. 实时化：有界队列、时钟、网络适配器、背压和恢复协议。

每一步都应保持旧行为绿色，并添加一条穿过输入、领域、输出与测试的新 tracer bullet。

13.7 性能报告怎么写

报告应包含硬件、操作系统、编译器、优化级别、数据规模、命令、预热、重复次数和统计量。布局基准提供本机 20 次交替顺序采样的中位数与 IQR；有价值的结论是“在指定环境与工作负载下，方案 A 的中位数比 B 低多少，并保持同一 checksum”，不是“二维数组永远更快”。

13.8 作为求职作品

README 的第一屏应回答问题、构建、运行和验证方式。面试讲解按“约束—设计—证据—限制”展开：为什么用值语义，为什么测试四个 seam，哪次测试发现真实问题，当前模型不能支持什么，下一步扩展如何保持契约。

展示失败和修复比声称“高性能框架”更可信。本项目中 Release 移除 assert 造成假绿色、ranges 的 const 视图编译失败、MiKTeX 依赖与模板兼容问题，都是工程判断的具体证据。

问题 13.1 面试检查点 请在五分钟内画出引擎数据流并说明每个所有权边界；解释为什么策略不直接修改组合；选择一个生产特性，描述其 tracer bullet 与验收事实；说明当前收益为何不能代表真实可交易表现。

13.9 练习

1. 增加固定每笔手续费，先写端到端手算测试，再实现。
2. 设计多资产权益快照，明确缺失行情和币种转换政策。
3. 为部分成交写状态机，列出非法状态转换。
4. 写一页性能实验报告，包含可复现命令和结论限制。

13.10 本章小结

贯穿项目的价值不在功能数量，而在端到端契约：不可信输入被验证，组件各守边界，结果可手算，限制被明示，演进可由 tracer bullet 驱动。这正是生产工程和面试系统设计共同需要的能力。

第十四章 量化 C++ 求职与入职

内容提要

- ❑ 把语言知识组织为代码、性能和系统设计能力
- ❑ 建立面试复盘与入职学习循环
- ❑ 用证据描述项目，不堆砌技术名词
- ❑ 识别不同量化岗位对 C++ 深度的差异

注 先修知识：完成贯穿项目并能解释其测试与限制。

Python 迁移边界：Python 经历是优势，但面试回答必须补充 C++ 的生命周期、成本模型和生产边界。

常见错误与诊断：准备材料若只有形容词没有命令、测试或数字，应删除形容词并补可复现证据。

14.1 先识别岗位

Quant Researcher 更重模型、实验和数据，但需要理解原生库与性能边界；Research Engineer 连接研究与生产，重视可复现数据管道、数值正确性和 Python/C++ 接口；Quant Developer 构建回测、执行、风险和基础设施；低延迟工程师进一步要求操作系统、网络、硬件计数器和并发内存模型。

不要用同一份准备清单覆盖所有岗位。阅读职位描述时，把要求映射为语言、算法、系统、金融领域和协作五列，再为每项准备“做过什么、证据是什么、还不知道什么”。

14.2 语言面试主线

高频问题不是孤立关键字，而是成本与契约：

- 对象生命周期、RAII、复制/移动与悬空；
- 值语义、引用、`const` 和智能指针选择；
- 容器布局、迭代器失效和复杂度；
- 模板、`concepts`、虚分派与 ABI；
- 异常安全、错误边界和 UB；
- 缓存、分配、benchmark 与 profiler；
- 数据竞争、锁、原子、队列和停止协议。

回答采用“定义—何时使用—代价—失败例子”。例如 `shared pointer`：引用计数共享生命周期；用于没有自然单一所有者的对象；成本包含控制块和原子计数；环会泄漏且不提供对象内部线程安全。

14.3 代码题

先澄清输入规模、所有权、错误和复杂度，再写小而正确的实现。选择 `vector` 作为默认序列，使用标准算法但不炫技。提交前检查空输入、整数溢出、索引边界、迭代器失效、比较器严格弱序和异常路径。

性能题不要立刻写 SIMD 或无锁队列。先定义指标和工作负载，给出基线表示，指出潜在缓存/分配成本，再说明如何测量。面试官通常更看重推理闭环。

14.4 系统设计题

量化系统设计可以沿数据生命线回答：来源、时间与排序、验证、存储、消费、状态、错误、回放、观测和容量。实时行情系统必须回答背压与断线恢复；回测平台必须回答数据版本、确定性、资源隔离与结果可追溯；订单系统必须回答幂等、状态机、风险门和审计。

明确一致性需求。交易前风险可能要求同步拒绝，研究统计可以最终一致；市场事件可能按交易所序列排序，而跨市场只有定义好的合并规则。不要用“Kafka + Redis + 微服务”替代需求分析。

14.5 项目如何写进简历

弱表述：“使用 C++、CMake 和多线程开发高性能回测框架。”它没有规模、行为和证据。

更可信的结构是：动作 + 系统边界 + 验证 + 测量。例如：

用 C++20 构建事件驱动回测内核，将 CSV 验证、策略、撮合和组合划分为四个公开测试 seam；以手算账本验证端到端收益，并建立 AoS/SoA 可重复基准，明确硬件与工作负载限制。

只有真实测量后才能加入吞吐或延迟数字，并保留复现实验命令。不要把教学模型称为生产级交易系统。

14.6 行为面试

准备 STAR 故事时优先选择有冲突和证据的工程事件：发现假绿色测试、反驳没有数据的优化、处理数据质量事故、在截止期内缩小范围、与研究者澄清可交易假设。说明你如何改变流程，而不仅是修了一行代码。

14.7 入职后的前三个月

前 30 天建立系统地图：构建、测试、部署、数据血缘、关键指标和术语；31–60 天完成小型端到端变更，理解审查与发布；61–90 天在证据支持下改善一个可靠性或性能问题。优先学习团队代码库的约定，不要把个人偏好当现代化改造。

问题 14.1 面试检查点 用两分钟解释本书项目；回答“为什么不用 shared pointer 管理所有事件”；设计行情队列的背压；给出一个带基线和限制的性能结论；说出当前项目最重要的三个非目标。

14.8 练习

1. 为目标职位做五列能力矩阵，并为每项附一条证据。
2. 录制五分钟项目讲解，删掉无法证明的形容词。
3. 完成一次 45 分钟代码题，复盘正确性、复杂度和沟通。
4. 为行情处理或回测平台画系统图，标注背压、恢复和观测点。

14.9 本章小结

求职准备应把 C++ 特性转化为生命周期、成本、正确性和系统边界的推理。项目陈述依靠可复现证据，系统设计沿数据生命线展开，入职后则从理解现有约束开始。

附录 A 工具链速查

A.1 本书验证环境

本项目在 Windows 上使用 MiKTeX 26.1、XeLaTeX、latexmk、CMake 3.29、GCC 13.2（MinGW-w64）验证。书稿主线代码以 Linux GCC/Clang 为目标，Windows 命令用于本机复现。

A.2 构建 C++

```
cmake -S . -B build-mingw -G "MinGW Makefiles" \  
      -DCMAKE_BUILD_TYPE=Release  
cmake --build build-mingw  
ctest --test-dir build-mingw --output-on-failure
```

Linux/Ninja 常用命令：

```
cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=Debug  
cmake --build build  
ctest --test-dir build --output-on-failure
```

诊断配置可加入 `-fsanitize=address,undefined -fno-omit-frame-pointer`。ThreadSanitizer 通常单独构建，不与 AddressSanitizer 混用。

A.3 构建 PDF

```
latexmk -xelatex -interaction=nonstopmode -halt-on-error \  
      -outdir=build main.tex
```

若 MiKTeX 为最小安装，可用新版 CLI 显式保证依赖：

```
miktex packages require elegantbook csquotes comment esint mwe \  
      enumitem caption footmisc multirow fancyvrb makecell lipsum \  
      hologo titlesec appendix biblatex pgf apptools bbding tcolorbox \  
      manfnt fancyhdr tocloft siunitx
```

本项目复制的 ElegantBook 4.7 在 MiKTeX 26.1 上遇到 `adorn` 的缺失间接依赖；本地类文件仅把问题集标题的两个装饰符替换为模板已加载的 `pifont` 字形。原模板目录未修改。

A.4 日志检查

```
rg -n "Undefined|undefined references|Missing character|Fatal error" \  
      build/main.log  
rg -n "Overfull|Underfull" build/main.log
```

未定义引用、缺字和致命错误必须修复。Overfull 需要逐条检查；Underfull 常是可接受的排版松弛，但不应忽略大段异常。

附录 B 术语表

术语	本书中的含义
ABI	二进制接口约定，包括调用、布局与符号兼容。
AoS / SoA	结构数组与数组结构，两种字段布局方式。
backpressure	下游跟不上时，系统对生产者施加的限速、阻塞或丢弃策略。
benchmark	在定义好的工作负载和环境中的测量性能的实验。
concept	C++20 对模板参数能力的编译期约束。
data race	未同步的并发冲突访问，至少一个写入；在 C++ 中属于 UB。
event	已验证、带时间和代码的市场事实。
fill	撮合后形成的成交事实，而不是订单意图。
happens-before	C++ 内存模型中保证效果可见与排序的关系。
iterator invalidation	容器操作使既有迭代器、引用或指针不再有效。
latency tail	延迟分布高百分位，如 p_{99} 与 $p_{99.9}$ 。
market event	规范化行情事件，包含时间、代码、价格与数量。
move	从可被转移资源的对象构造/赋值； <code>std::move</code> 仅转换值类别。
order intent	策略产生的买卖意图，尚不是成交。
ownership	谁负责保持对象存活并最终释放其资源。
PortfolioSnapshot	组合通过公开接口暴露的只读持仓、现金与权益值。
profiling	定位程序时间、分配或硬件事件分布的测量。
RAII	将资源取得与对象初始化绑定、释放与析构绑定。
seam	通过公开接口观察行为的测试边界。
slippage	决策或参考价格与实际模拟/成交价格之间的差异。
tracer bullet	从输入到输出贯通、可独立验证的最小纵向切片。
UB	未定义行为；语言标准不再约束程序结果。
value semantics	对象复制后成为具有相同含义的独立值。
view	通常不拥有元素、惰性描述遍历方式的范围适配器。
zero-cost abstraction	不使用不付费，使用抽象通常不劣于对应手写底层机制的设计原则。

附录 C 练习答案与思路

本附录给出检查点而非唯一实现。编码题应以可编译代码、行为测试和复杂度说明为最终答案。

C.1 第 1 章

自测答案

1. `#include` 在预处理阶段处理，不是以分号结束的 C++ 语句；它让当前翻译单元看见 `iostream` 中的声明。
2. `main` 是程序入口名，空圆括号表示本版本不接收参数，花括号包围函数体；状态 0 表示成功，非零表示失败。
3. `std::` 指定标准库命名空间，`<<` 把右侧文本送入输出流，双引号界定字符串字面量，`\n` 换行，分号结束语句。
4. 流水线是预处理、编译、链接和运行；`-c` 在目标文件处停止，因此能证明编译通过，却不能证明链接成功。
5. 缺分号是编译失败；只有声明没有定义是链接失败；打印启动错误并返回 2 是运行期失败。修复分别恢复语法、加入定义、补齐启动条件。
6. CMake 用 `-S` 指源目录、`-B` 指构建目录。删除旧构建树后从不存在的 `build/` 成功配置、构建和运行，才是干净构建证据。
7. Windows 可执行文件带 `.exe`，路径分隔和编译器警告选项不同，多配置生成器还包含配置目录；预处理、编译、链接、运行的阶段模型不变。

编码任务反馈

1. 完整文件见 `code/ch01/first_program.cpp` 与 `code/ch01/CMakeLists.txt`。直接 GCC 和干净 CMake 构建都应输出 `quant-cpp-ready` 并返回 0。
2. 三个失败源依次是 `missing_semicolon.cpp`、`missing_definition.cpp` 与 `startup_failure.cpp`，均位于 `code/ch01/failures/`；验收类别分别是编译期缺分号、链接期缺定义、运行期消息加状态 2。
3. `//` 到行末结束，`/* ... */` 可以跨行但不能嵌套；两种注释都不应改变输出。删除分号应使编译失败。恢复后重跑 `ctest -R ch01_`，五个第 1 章行为测试应全部通过。

C.2 第 2 章

1. 初始化在对象创建时给出初始状态；赋值修改已存在对象。局部 `int x`；没有可靠初值，`int x{}`；值初始化为零。字符字面量用单引号，字符串字面量用双引号；可运行演示见 `code/ch02/objects_and_literals.cpp`。完整筛选答案见 `code/ch02/market_filter.cpp`，给定五行输入应输出 `selected=2 rejected=3 buy_notional=1035.75 average=517.875`。
2. `auto` 仍在编译期推导出固定类型；当位宽或领域含义重要时显式写出类型。`5 / 2` 先做整数除法得到 2，`5.0 / 2` 得到 2.5；平均值分母可写成 `static_cast<double>(selected)`。
3. `a && b` 在 `a` 为假时不求值 `b`。`switch` 漏写 `break` 会贯穿后续标签；`default` 接住未识别方向。已知迭代次数用 `for`，由状态决定终止用 `while`。
4. 风险预算题的完整答案见 `code/ch02/exercises/risk_budget_solution.cpp`；3200 按每笔 750 扣减四次后剩 200。计数循环的独立答案见 `code/ch02/for_scope.cpp`。若循环体不扣减预算，条件永不改变，会形成无限循环。

- code/ch02/failures/narrowing.cpp 应在编译阶段得到“列表初始化拒绝窄化转换”类别。可把接收对象改为 double；若业务确实需要整数，先定义截断、四舍五入或拒绝策略，再显式转换。

C.3 第3章

自测答案

- 声明给出返回类型、名称和参数表并以分号结束；定义给出匹配接口与函数体；调用用实参创建参数对象并取得结果。定义的右花括号后不加函数声明式分号。
- 按值或引用参数在调用期间有效，局部对象在离开函数时销毁；返回值交回调用者。早返回在满足条件时立即结束本次调用，跳过后续语句。
- 按值允许修改独立参数而不影响调用者；const T& 只读借用；T& 可修改调用者；const T* 是可空的只读观察，检查不为 nullptr 后才能解引用。
- 重载靠参数数量或类型区分，并要求调用时存在唯一最佳候选。只改变返回类型无法总由实参确定选择，因此不能形成可调用的重载集合。
- 每个 .cpp 连同包含的声明形成独立翻译单元并编译为目标文件；链接器把目标文件与库组合为可执行文件。头文件自身不会自动提供实现。
- include guard 让同一头文件在一个翻译单元重复包含时只展开一次。项目头文件用双引号配合目标 include 路径；命名空间降低项目名称与其他库冲突的风险。
- 缺失定义与重复定义都在各翻译单元编译后由链接器报告；返回局部对象引用可由高警告级别在编译时指出，其后读取是未定义行为，不运行来验收结果。

编码任务反馈

- 完整多文件答案位于 code/ch03/include/quant/ch03/market_math.hpp、code/ch03/src/market_math.cpp 与 code/ch03/src/main.cpp；独立 code/ch03/CMakeLists.txt 应从空构建树得到 notional=4010.00 quantity=40 vwap=100.25。
- 完整可运行答案如下；输出应为 rate=0.0015 basis_points=15，并证明调用者值不变：

```

1  #include <iostream>
2
3  double basis_points(const double& rate) {
4      return rate * 10000.0;
5  }
6
7  int main() {
8      const double rate{0.0015};
9      const double converted{basis_points(rate)};
10
11     std::cout << "rate=" << rate << " basis_points=" << converted << '\n';
12     return 0;
13 }
```

- 三个故障源码位于 code/ch03/failures/。缺失与重复定义分别匹配链接器的 undefined 与 multiple-definition 类别；悬空引用匹配返回局部对象引用的编译诊断。恢复方式分别是补上唯一定义、删除额外定义、改为按值返回或证明被引用对象寿命足够长。

C.4 第 4 章

1. filter 代码和 side 后循环累计 $price \times quantity$ ，分母累计 quantity。
2. 小规模连续扫描可能胜出；报告交叉点、构建成本和查询次数。
3. 扩容后旧引用悬空；预留容量或用索引并在操作后重新取元素。
4. 使用容差或稳定归约；若要求逐位确定，应固定顺序并避免不受控并行归约。

C.5 第 5 章

1. ID 若决定订单身份应是值字段；追踪批次等上下文可放外部 envelope。
2. 返回成交与剩余量，且任何成交量不超过订单和事件可用量。
3. 快照接收价格映射和估值时刻，并返回完整性状态；禁止静默把缺价当零。
4. 按不变量拆分，例如订单生命周期与风险检查，不按“工具方法”随意切。

C.6 第 6 章

自测答案

1. 构造成功后对象生命周期开始，析构完成后结束；存储中的旧比特不会让已析构对象重新存在。自动对象按逆构造顺序销毁。
2. `std::move` 只把表达式转换为可选择移动操作的值类别。移动后的源对象仍可析构、可赋值，但除非类型契约另有保证，不能假定保留原值。
3. 引用和裸指针通常只借用；`unique_ptr` 是可转移的唯一所有者；`shared_ptr` 让多个参与者共同决定寿命。直接值仍是最简单的默认所有权。
4. C 数组退化为指针后不再携带长度。`new[]` 建立的数组必须由 `delete[]` 释放；与标量释放形式混用是未定义行为。
5. RAII 让栈展开和所有提前离开路径都触发析构；Rule of Zero 把释放交给可靠成员，避免外层类型手写一组容易不一致的复制、移动和析构操作。
6. `weak_ptr` 不增加强计数，因此可打破共享环；`shared_ptr` 只协调控制块寿命，不会自动同步被管理对象的字段。
7. ASan 报告同时给出分配、释放和非法访问位置。按这三处重建生命周期图，再恢复单一明确的所有者和不越界的借用期。

编码练习反馈

1. 复制/移动轨迹的完整答案及精确输出见 `code/ch06/lifetime_trace.cpp` 与 `code/ch06/expected/lifetime_trace.txt`。
2. 引用、可空观察、C 数组和手动释放的可运行答案见 `code/ch06/borrowing_and_legacy.cpp`。
3. RAII 文件答案见 `code/ch06/raii_file.cpp`：输出流析构后输入流读到两行，输入流析构后文件才删除。
4. 所有权输出任务见 `code/ch06/ownership_choices.cpp`；答案必须与对应 `expected` 文件逐字一致。
5. 故意失败源码位于目录 `code/ch06/failures/` 的 `use_after_free.cpp`。ASan 应非零退出并报告 `heap-use-after-free`，不验收未定义的输出值。

C.7 第 7 章

1. 三个输入均应返回失败，错误包含行号和类别；测试不依赖内部解析步骤。
2. 结构化错误便于程序判断和本地化，字符串简单但难稳定消费。
3. 若调用者需要统计拒绝原因，返回枚举/结果；空值只适合原因不重要。
4. 例：解析失败率、队列深度、任务耗时；失败日志记录路径摘要和任务 ID。

C.8 第 8 章

1. 单独 build 目录加入 sanitizer flags；全部测试绿色且无 sanitizer 报告才通过。
2. 端到端权益应偏离 10100，并以“最终权益”行为消息失败。
3. 只调用 run 并检查结果，不读取 Portfolio 的内部 map。
4. GCC/Clang Debug、sanitizer、Release smoke 各覆盖不同故障；固定硬件另跑性能。

C.9 第 9 章

1. 在多个独立进程中重复 20 次交替顺序采样，比较中位数与 IQR；不要删除异常值而不说明。
2. 对象缩小可能提高扫描密度，实际收益依赖读取字段与缓存层次。
3. 若分配样本占比很低，不引入池；可先 reserve 并重测。
4. 每线程独立 cache-line-aware 累计，join 后单线程归约。

C.10 第 10 章

1. 保存每次耗时，排序取中位数和四分位；同时验证每次 checksum。
2. 假设应可反驳，例如“解析占 CPU 样本超过 40%”；profile 后接受或拒绝。
3. 指标必须带负载，如每秒百万事件下 $p_{99} < X$ ，且错误/丢弃为零。
4. 需要显著业务容量或成本收益、可维护封装、回归基准和回退路径。

C.11 第 11 章

1. 按资产分区输入，每线程只写自己的结果 map，join 后合并。
2. 阻塞传播背压；丢最新损失新信息；覆盖最旧损失历史，均需指标和业务许可。
3. 先停接收，再通知消费者排空/取消，唤醒阻塞，join，最后释放依赖。
4. TSan 报告定位冲突访问；修复应建立所有权分片或明确同步，而非加随机 sleep。

C.12 第 12 章

1. 容差来自收益报告最小单位和算法误差，使用绝对 + 相对组合。
2. Welford 同时更新均值与平方差，避免大数相减；与两遍算法用高精度参考比较。
3. 规定二维/一维 shape、float64、C contiguous、借用期、异常映射和批量返回。
4. 若时间已在 NumPy 原生核，重写外层无益；若 Python 逐元素循环主导，批量 C++ 值得原型。

C.13 第 13 章

1. 手算权益应再减手续费；测试先红，再让成交/组合一次性记录费用。
2. 价格映射带时刻与币种，缺价返回不完整或错误，不能默认为零。
3. 典型状态为 new、partially filled、filled、cancelled、rejected；终态不能再成交。
4. 报告必须列环境、命令、数据、重复、统计、checksum 和结论适用范围。

C.14 第 14 章

1. 每项证据可链接代码、测试、基准或事故复盘；“了解”不是证据。
2. 删除“高性能、生产级”等无测量词，保留边界、行为和限制。
3. 复盘应区分理解、算法、编码、测试与沟通时间，选一个改进动作。
4. 图中至少含输入、排序、队列、消费者、状态、存储、恢复和指标/告警。

附录 D 推荐阅读与 30 天路线

D.1 推荐阅读

- Bjarne Stroustrup, *A Tour of C++*: 建立现代语言全貌。
- Scott Meyers, *Effective Modern C++*: C++11/14 机制仍是所有权与移动语义基础。
- Anthony Williams, *C++ Concurrency in Action*: 线程、内存模型与并发结构。
- Agner Fog 的优化手册: 指令、微架构与测量, 阅读时结合目标硬件更新资料。
- Brendan Gregg, *Systems Performance*: 系统观测、方法与工具。
- John Lakos 等, *Large-Scale C++*: 依赖、组件和大型代码库边界。
- cppreference: 核对语言与标准库语义; 不要只依赖博客摘要。
- C++ Core Guidelines: 作为审查清单, 结合团队规则而非机械执行。

D.2 第 1 周: 语言与所有权

日	任务
1	配置 CMake/GCC/Clang, 构建第 1 章示例, 解释编译与链接。
2	完成类型、初始化与窄化练习。
3	练习引用、 <code>const</code> 、值类别与移动。
4	用 RAII 封装文件和计时器。
5	比较值成员、 <code>unique pointer</code> 与 <code>shared pointer</code> 。
6	完成 <code>vector</code> 、算法和 <code>ranges</code> 数据管道。
7	复盘四章面试检查点, 不看书口述答案。

D.3 第 2 周: 接口与可靠性

第 8–9 天设计事件、订单、成交和组合不变量; 第 10 天练习模板与 `concepts`; 第 11 天完成 CSV 错误测试; 第 12 天运行四个 `seam`; 第 13 天添加 `sanitizer` 构建; 第 14 天从干净目录重新构建并写一页架构说明。

D.4 第 3 周: 性能与并发

第 15 天学习缓存与布局; 第 16 天重复布局基准并做统计; 第 17 天使用 `profiler`; 第 18 天复习浮点误差; 第 19 天实现稳定归约; 第 20 天画出并发所有权图; 第 21 天实现一个有界队列设计文档并说明背压, 不急于写无锁代码。

D.5 第 4 周: 项目与求职

第 22–24 天为回测项目增加一个经 TDD 驱动的真实特性; 第 25 天完善 README 和复现命令; 第 26 天写性能报告; 第 27 天准备语言问答; 第 28 天做系统设计模拟; 第 29 天录制项目讲解; 第 30 天完成一次全流程模拟面试并制定下一月计划。

D.6 完成判据

30 天后，不以“读完章节”为成功，而以以下证据验收：能从干净目录构建；能解释四个 seam；能手算一个回测结果；能展示一次红绿循环；能提供带环境和限制的性能报告；能画出系统边界、背压和停止协议；能诚实陈述项目非目标。

参考文献

- [1] *cppreference.com*. URL: <https://en.cppreference.com/> (visited on 07/11/2026).
- [2] Brendan Gregg. *Systems Performance: Enterprise and the Cloud*. 2nd ed. Addison-Wesley, 2020.
- [3] Scott Meyers. *Effective Modern C++*. O'Reilly Media, 2014.
- [4] Bjarne Stroustrup. *A Tour of C++*. 3rd ed. Addison-Wesley, 2022.
- [5] Bjarne Stroustrup and Herb Sutter. *C++ Core Guidelines*. URL: <https://isocpp.github.io/CppCoreGuidelines/> (visited on 07/11/2026).
- [6] Anthony Williams. *C++ Concurrency in Action*. 2nd ed. Manning, 2019.

模板与许可说明

本书使用 ElegantBook 模板排版，项目主页如下：

<https://github.com/ElegantLaTeX/ElegantBook>

类文件依照 LaTeX Project Public License 1.3c 或更高版本发布。模板许可文本随项目保存在独立文件中：

ELEGANTBOOK-LICENSE